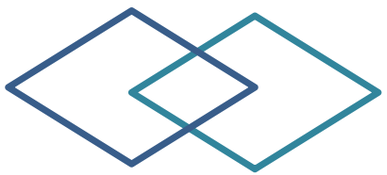


第7章

串口通信



本章知识与能力要求

- 理解和掌握通信的基本概念；
- 了解常见的串行通信接口；
- 熟悉标准外设库和HAL库中有关USART的库函数；
- 掌握STM32F103微控制器USART标准外设库异步模式的配置方法；
- 掌握USART查询方式、中断方式以及DMA方式三种方式的应用；
- 掌握基于标准外设库开发USART应用的方法；
- 掌握基于HAL库开发USART应用的方法。

第 7 章 串口通信



7.1 通信概述

7.2 异步串行通信

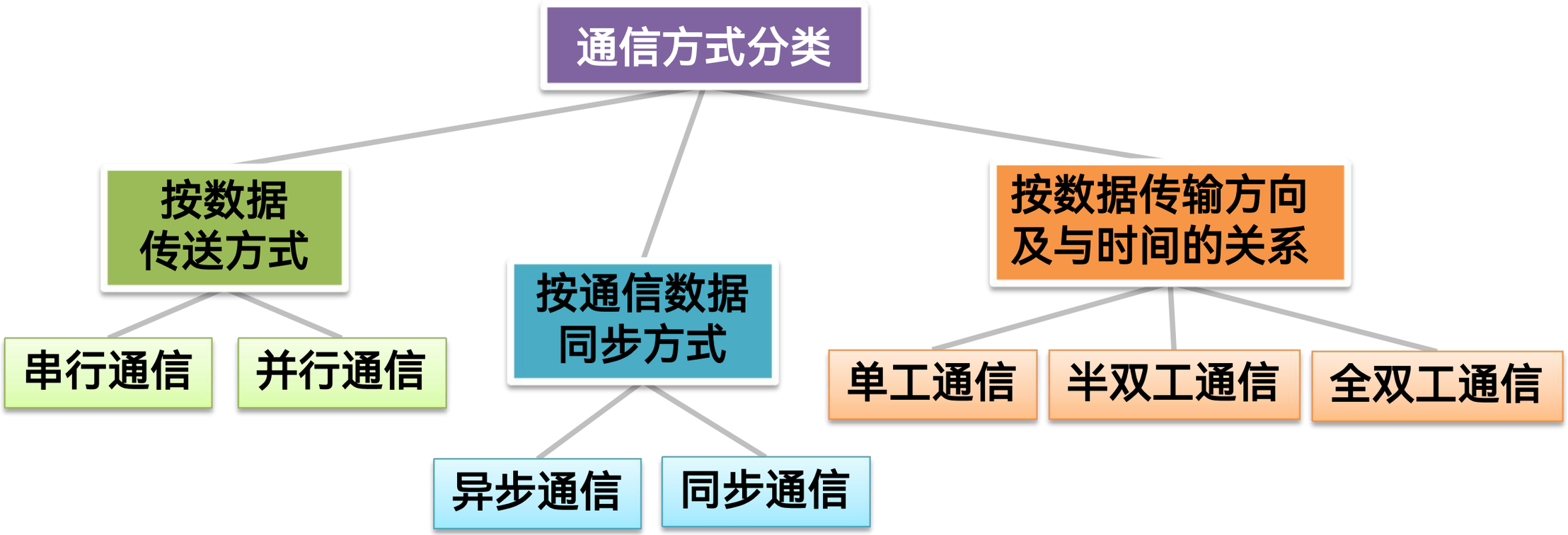
7.3 STM32的USART模块

7.4 USART模块的标准外设库接口函数及应用

7.5 USART模块的HAL库接口函数及应用

7.6 编程思想之自定义串口通信协议

7.1 通信概述

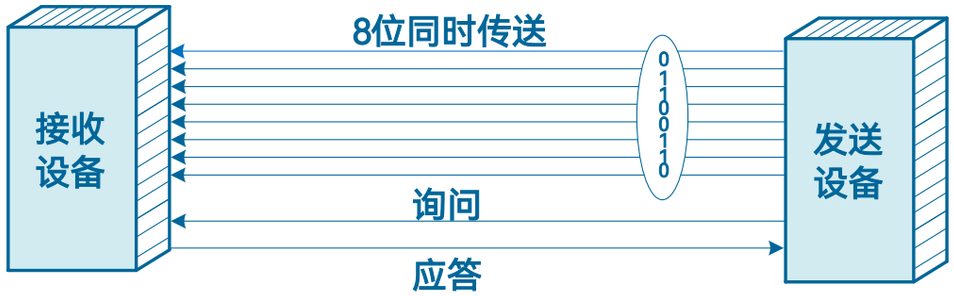

通信方式分类
按数据
传送方式
串行通信
并行通信
按通信数据
同步方式
异步通信
同步通信
按数据传输方向
及与时间的关系
单工通信
半双工通信
全双工通信

7.1 通信概述

串行通信与并行通信

串行通信(Serial Transmission），是指通过一根数据线或少量数据线（少于8根）将数据一位一位地按顺序依次传送。

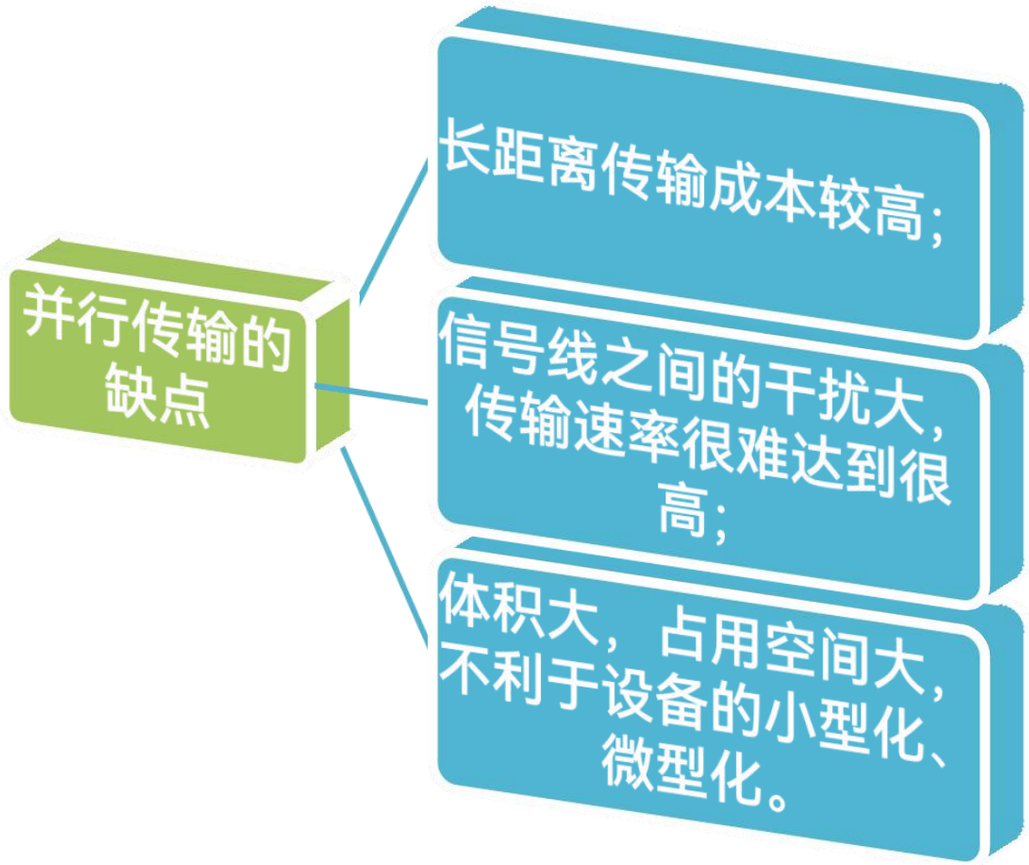




并行通信(Parallel Transmission），是指用多条数据线同时传送多位数据，通常以8位、16位、32位等数据位传送。

7.1 通信概述

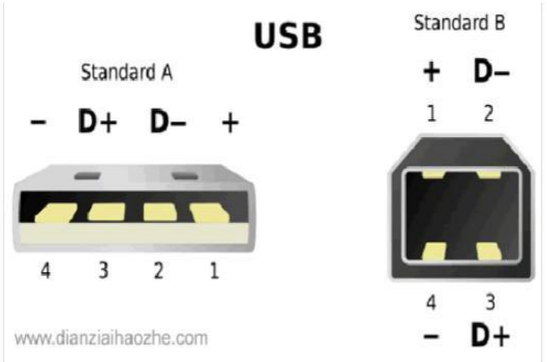
并行通信一次可传输多个数据位的数据，并行通信传输数据量是串行传输的数倍，如早期的并行接口的硬盘、打印机，采用PCI(Peripheral Component Interconnect，外设部件互连标准）并行接口的显卡等。



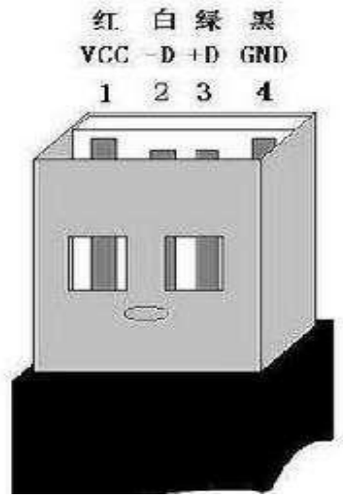
计算机内部的总线结构，即CPU与内部寄存器及接口之间采用并行传输。

7.1 通信概述

串行通信是目前最为流行的一种通信方式，如U盘以及采用USB接口的设备、工业上常用的RS-485和RS-232等，都是串行通信设备。



USB的串行通信接口



U盘的串行通信接口

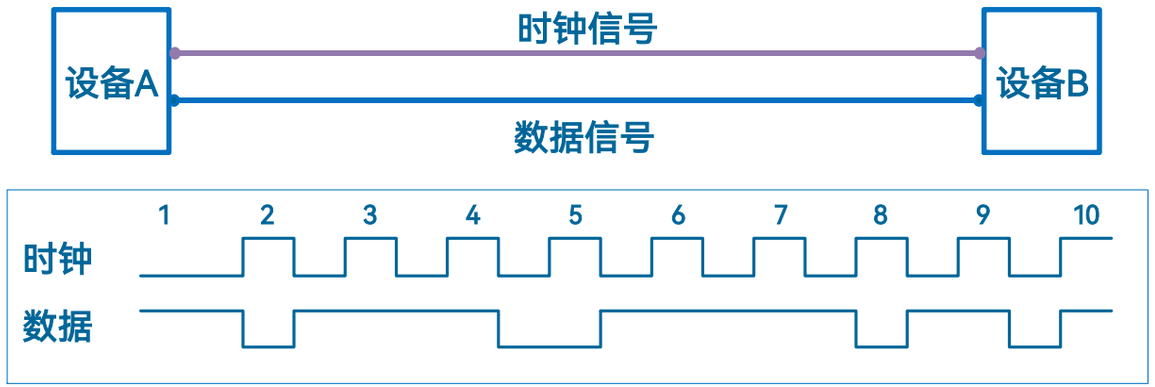
计算机网络中所使用的传输方式均是串行传输，单片机与外设之间也大多采用各类串行接口，比如USART、USB、IIC、SPI等。

7.1 通信概述

同步通信与异步通信

同步通信是连续串行传送数据的通信方式，要求收发双方的时钟必须保持严格的同步。

特点： 输速率较高



异步通信在发送的有效数据中增加一些用于同步的控制位，比如开始位和停止位等，数据以字符为单位组成数据帧进行传送，收发双方需约定数据的传输速率。

特点：

- 传输效率较低；
- 通信设备实现简单、成本低。

7.1 通信概述

单工、半双工、全双工通信

根据数据传输方向以及与时间的关系，串行通信可以分为单工、半双工和全双工三种通信方式。

通信方式	描述	举例
单工	任何时刻都只能单方向通信的工作方式，即一端固定为发送端，另一端固定为接收端	灯塔、广播和电视
半双工	两个设备均可以收发数据，但不能在同一时刻进行	对讲机
全双工	两个设备可以同时收发数据	手机

7.1 通信概述

通信传输速率

波特率

每秒传输的二进制位数，单位为比特每秒（bit/s，bps），是衡量串行数据传输速度快慢的指标。

字符速率

每秒所传输的字符数：

波特率=字符速率×每个字符包含的位数

举例

字符速率为120字符/秒，若每个字符包含10位（1个起始位，7个数据位，1个校验位，1个结束位），则波特率为

10位/字符×120字符/秒 = 1200bps。

常用的串口传输速率（波特率）有1200、2400、4800、9600、19200和115200等。

7.2 异步串行通信

目 录



7.2.1 异步串行通信协议

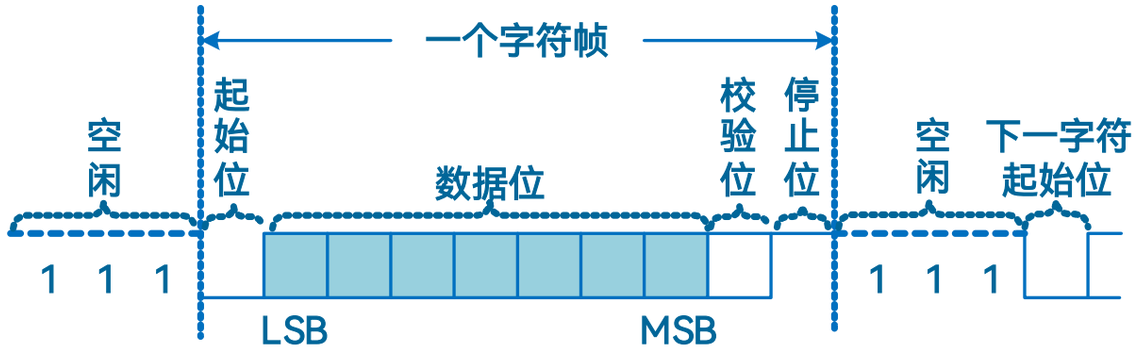


7.2.2 异步串行通信接口

7.2.1 异步串行通信协议

通信传输速率和数据帧格式一起称为通信协议。

异步串行通信标准的数据帧由起始位、数据位、校验位、停止位四部分组成。数据传输速率为50、75、100、150、300、600、1200、2400、4800、9600、19200和38400波特。



起始位：占一位，位于数据帧的开头，以逻辑“0”表示传输数据的开始。

数据位：要发送的数据，数据长度可以是5 ~ 8位。

校验位：占一位，用于检测数据是否有效。

停止位：一帧传送结束的标志，根据实际情况定，可以是1、1.5或2位。

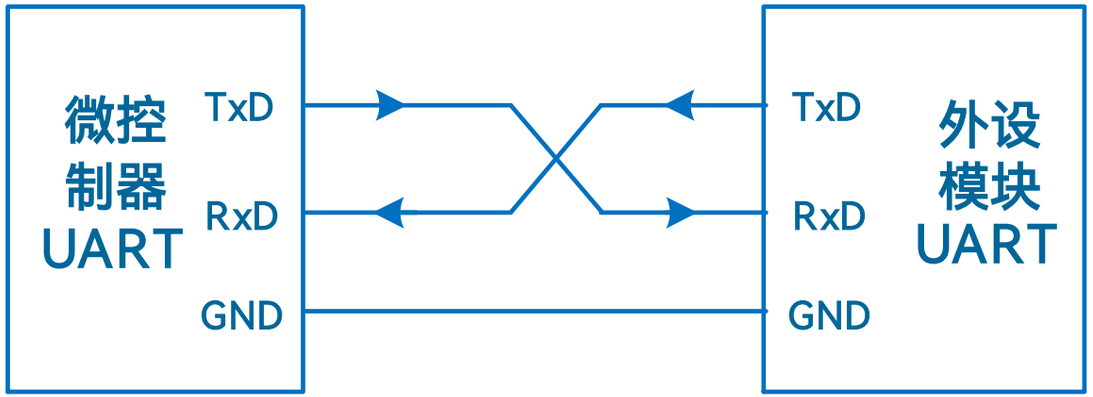
空闲位：数据传输完毕，用“1”表示当前线路上没有数据传输。

7.2.1 异步串行通信接口

UART（Universal Asynchronous Receiver Transmitter，通用异步收发传输器）是一个全双工通用异步串行收/发模块，主要用于打印程序调试信息、上位机和下位机的通信以及ISP程序下载等场合。

UART至少需要两根数据线用于通信双方进行数据双向同时传输，最简单的UART接口由TxD、RxD、GND共3根线组成。其中，TxD用于发送数据，RxD用于接收数据，GND为信号地线，通过交叉连接实现两个芯片间的串口通信。

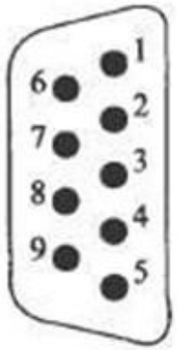
- RxD：数据输入引脚，数据接受。
- TxD：数据输出引脚，数据发送。



7.2.1 异步串行通信接口

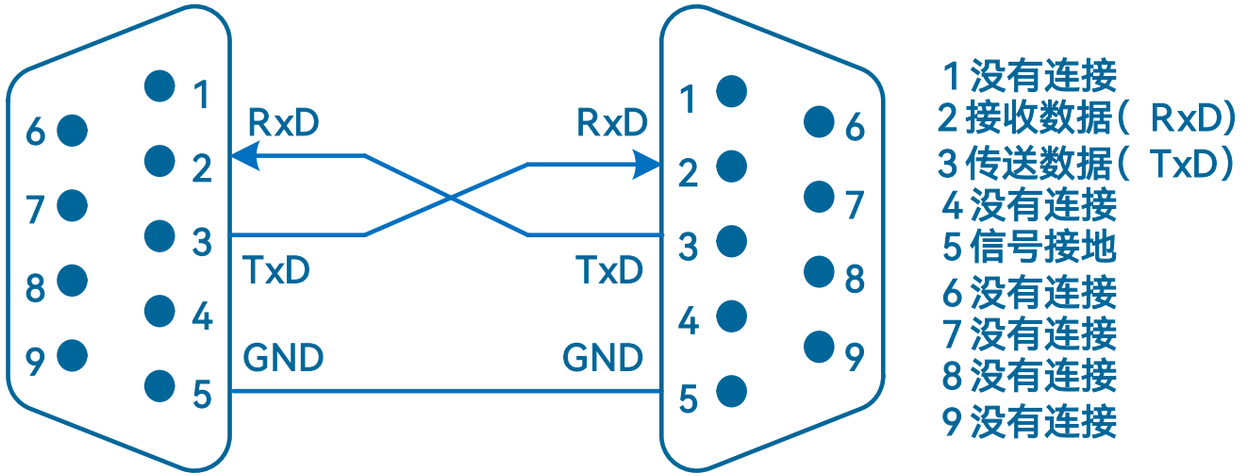
RS-232是美国电子工业协会(EIA, Electronic Industry Association)制定的串行通信物理接口标准，规定了相应的电气特性和物理特性，通常采用DB-9或DB-25的形式，以DB-9最为常见。



外形	针脚号	符号	功能
	1	DCD	数据载波检测，输入
	2	RxD	接收数据，输入
	3	TxD	发送数据，输出
	4	DTR	数据终端准备就绪，输出
	5	GND	信号地
	6	DSR	数据设备准备就绪，输入
	7	RTS	请求发送，输出
	8	CTS	清除发送，输入
	9	RI	振铃指示，输入

7.2.1 异步串行通信接口

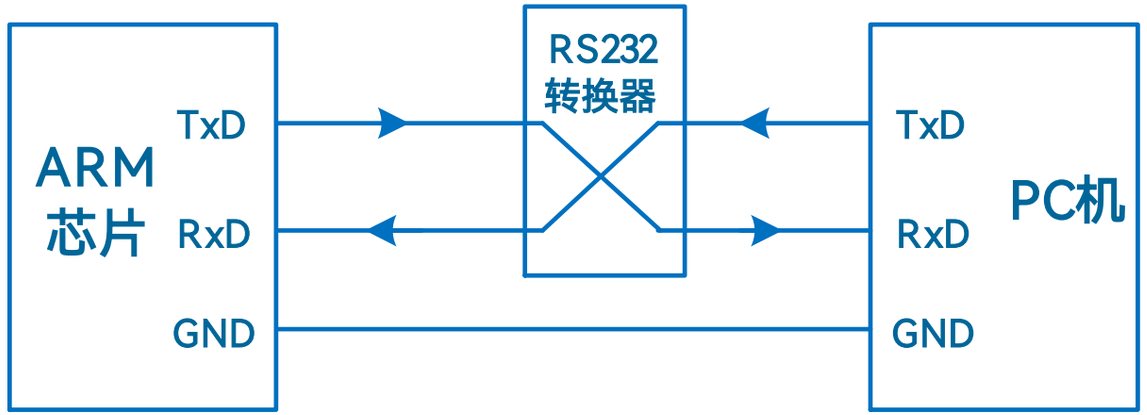
对可靠性要求不高的场合，DB-9通常采用三线制串口，仅需发送（Tx）、接收(Rx）和地（GND）三条线，即可实现全双工通信，进行最基本的数据收发传送，最高传输速率可达20kbps



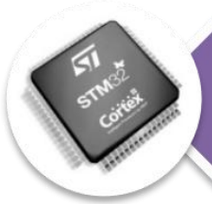
7.2.1 异步串行通信接口

RS-232为异步串行通信接口，其电气标准采用负逻辑，不能与TTL电平设备直接相连，而微控制器中的UART采用的是TTL电平标准，因此，RS-232与微控制器相连时必须进行电平转换。

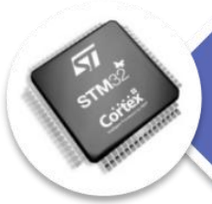
-RxD：数据输入引脚，数据接受。
-TxD：数据输出引脚，数据发送。



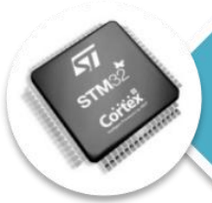
7.3 STM32的USART模块



7.3.1 USART内部结构



7.3.2 USART接口

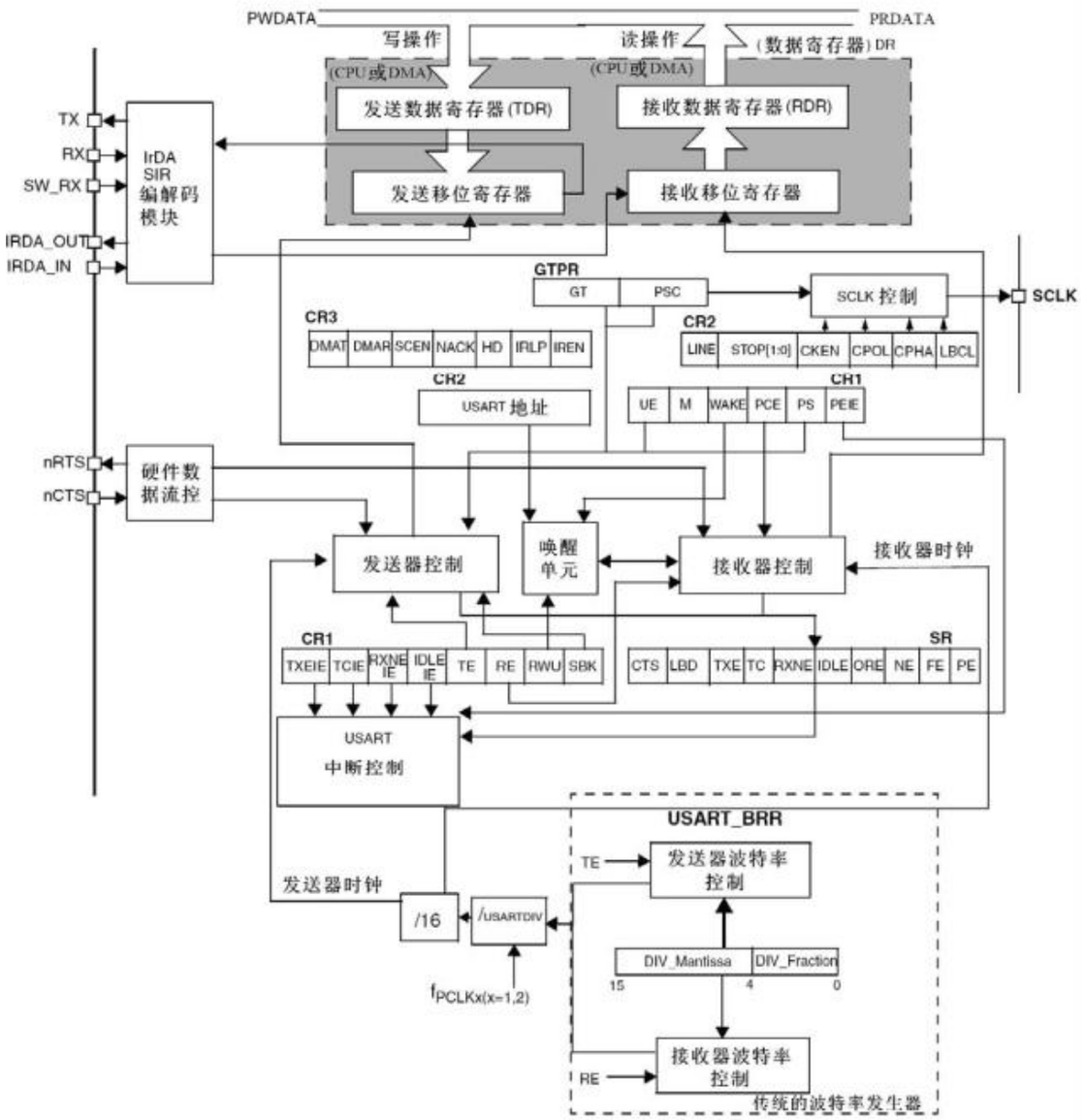


7.3.3 USART编程模式

7.3.1 USART内部结构

USART内部功能实现由三部分构成:





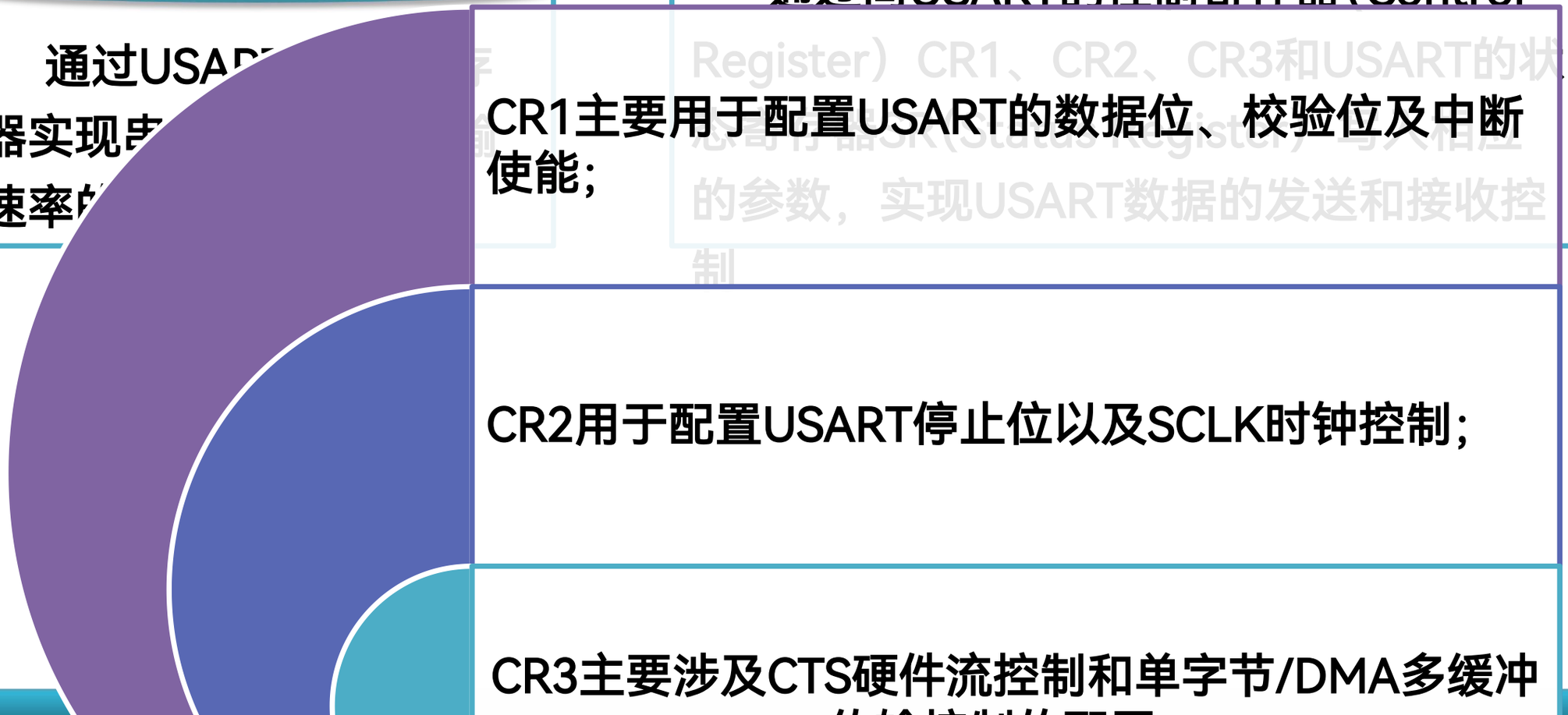
7.3.1 USART内部结构

波特率发生器

通过USAR
器实现串
速率的

发送器/接收器控制单元

通过向USART的控制寄存器（Control Register）CR1、CR2、CR3和USART的状态寄存器SR（Status Register）写入相应的参数，实现USART数据的发送和接收控制



CR1主要用于配置USART的数据位、校验位及中断使能；

CR2用于配置USART停止位以及SCLK时钟控制；

CR3主要涉及CTS硬件流控制和单字节/DMA多缓冲

7.3.1 USART内部结构

USART状态寄存器SR各bit位所代表的含义

bit位	寄存器名称	功能描述
31-10	保留，硬件强制设置为“0”	
9	CTS	CTS标志 0：nCTS状态线上没有变化；1：nCTS状态线上发生变化。
8	LBD	LIN断开检测标志(LIN break detection flag) 0：没有检测到LIN断开；1：检测到LIN断开。
7	TXE	发送数据寄存器空(Transmit data register empty) 0：数据还没有被转移到移位寄存器；1：数据已经被转移到移位寄存器。
6	TC	发送完成(Transmission complete) 0：发送还未完成；1：发送完成。
5	RXNE	读数据寄存器非空 (Read data register not empty) 0：数据没有收到；1：收到数据，可以读出。

7.3.1 USART内部结构

USART状态寄存器SR各bit位所代表的含义

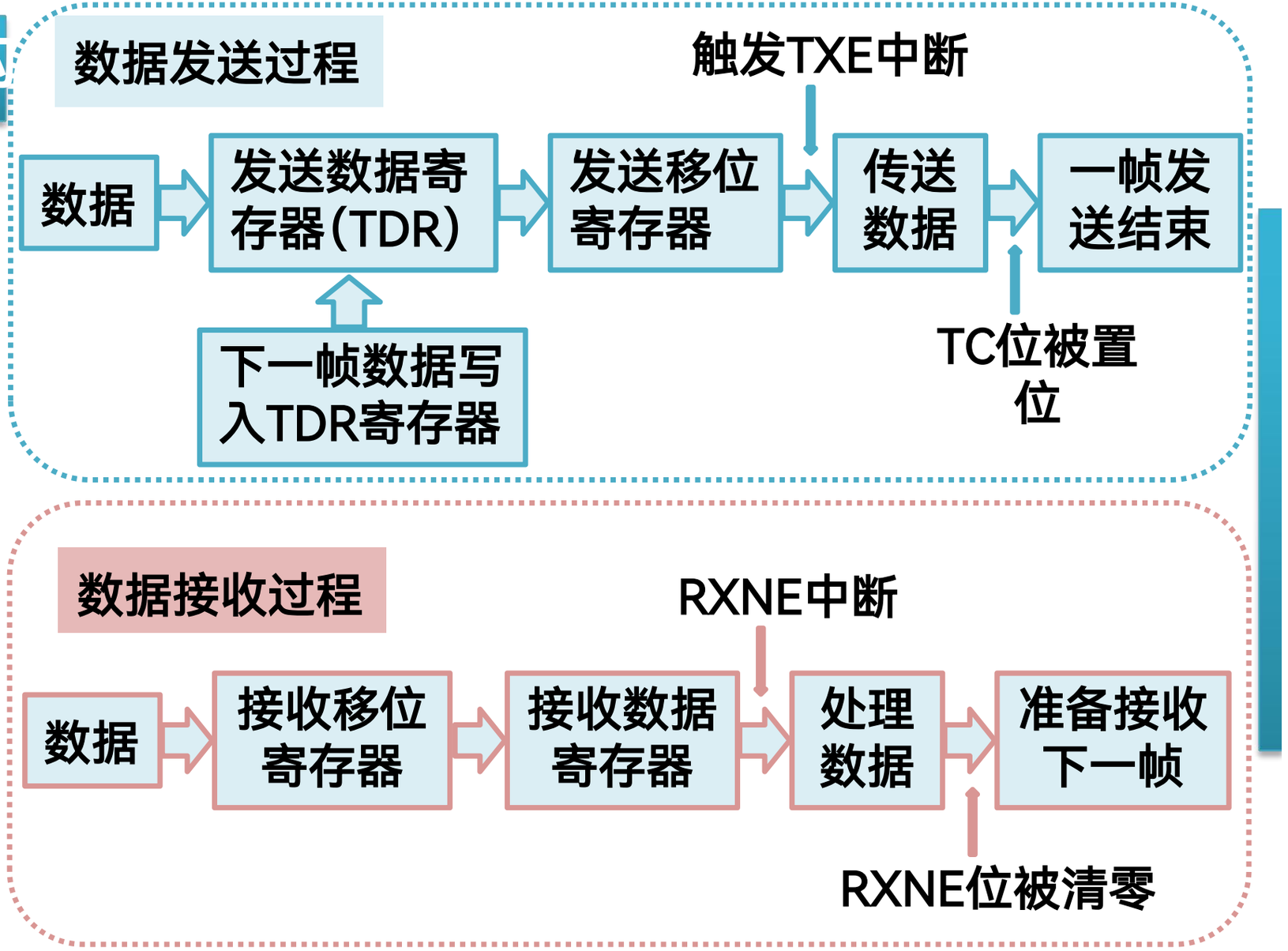
bit位	寄存器名称	功能描述
4	IDLE	监测到总线空闲(IDLE line detected) 0：没有检测到空闲总线；1：检测到空闲总线。
3	ORE	过载错误(Overrun error) 0：没有过载错误；1：检测到过载错误
2	NE	噪声错误标志(Noise error flag) 0：没有检测到噪声；1：检测到噪声。
1	FE	帧错误(Framing error) 0：没有检测到帧错误；1：检测到帧错误或者break符。
0	PE	校验错误(Parity error) 0：没有奇偶校验错误；1：奇偶校验错误。

7.3.1 USART内部结构

数据收发寄存器单元

组成：

- 发送数据寄存器（TDR）
- 接收数据寄存器（RDR）
- 发送移位寄存器
- 接收移位寄存器



7.3.2 USART接口

STM32F103大容量产品有5个串行通信接口，USART4和USART5为2个通用异步收发器，不支持同步模式，只有异步通信功能。

USART模式	USART1	USART2	USART3	USART4	USART5
异步模式	ï（支持）	ï	ï	ï	ï
硬件流控制	ï	ï	ï	ï（不支持）	ï
多缓存通信(DMA)	ï	ï	ï	ï	ï
多处理器通信	ï	ï	ï	ï	ï
同步	ï	ï	ï	ï	ï
智能卡	ï	ï	ï	ï	ï
半双工(单线模式)	ï	ï	ï	ï	ï
IrDA	ï	ï	ï	ï	ï
LIN	ï	ï	ï	ï	ï

7.3.2 USART接口

USART异步模式至少需要两个引脚：Rx(接收数据引脚）和Tx(发送数据引脚），使用了复用I/O功能。

总线	USARTx	复用功能	USART1_REMAP=0	USART1_REMAP=1	GPIO配置
APB2总线	USART1	USART1_TX	PA9	PB6	复用推挽输出
		USART1_RX	PA10	PB7	浮空输入
APB1总线	USART2	USART2_TX	PA2	PD5	复用推挽输出
		USART2_RX	PA3	PD6	浮空输入
	USART3	USART3_TX	PB10	PC10	复用推挽输出
		USART3_RX	PB11	PC11	浮空输入

7.3.3 USART编程模式

轮询模式

- CPU不断地查询I/O设备是否准备就绪，如果准备就绪就发送，否则提示超时错误；
- 会占用CPU的大量时间，效率低。

中断方式

- 通过中断请求线，在I/O设备准备就绪时向CPU发出中断请求，CPU中止正在进行的工作转向处理I/O设备的中断事件
- 中断方式相比轮询方式效率较高。

DMA方式

- 直接存储器传送，不经过CPU直接在内存和外设之间进行批量数据交换，适用于高速大批量成组数据的传输；
- 满足高速I/O设备的传输要求，有利于提高CPU的利用率。

7.3.3 USART编程模式

轮询方式

通过定期或循环查询USART状态寄存器SR各位的状态，编写应用程序对接收或发送的数据进行处理，有两种方式实现对USART相关寄存器的操作：

寄存器开发方式

在编程代码中直接配置这些功能寄存器

基于库的开发方式

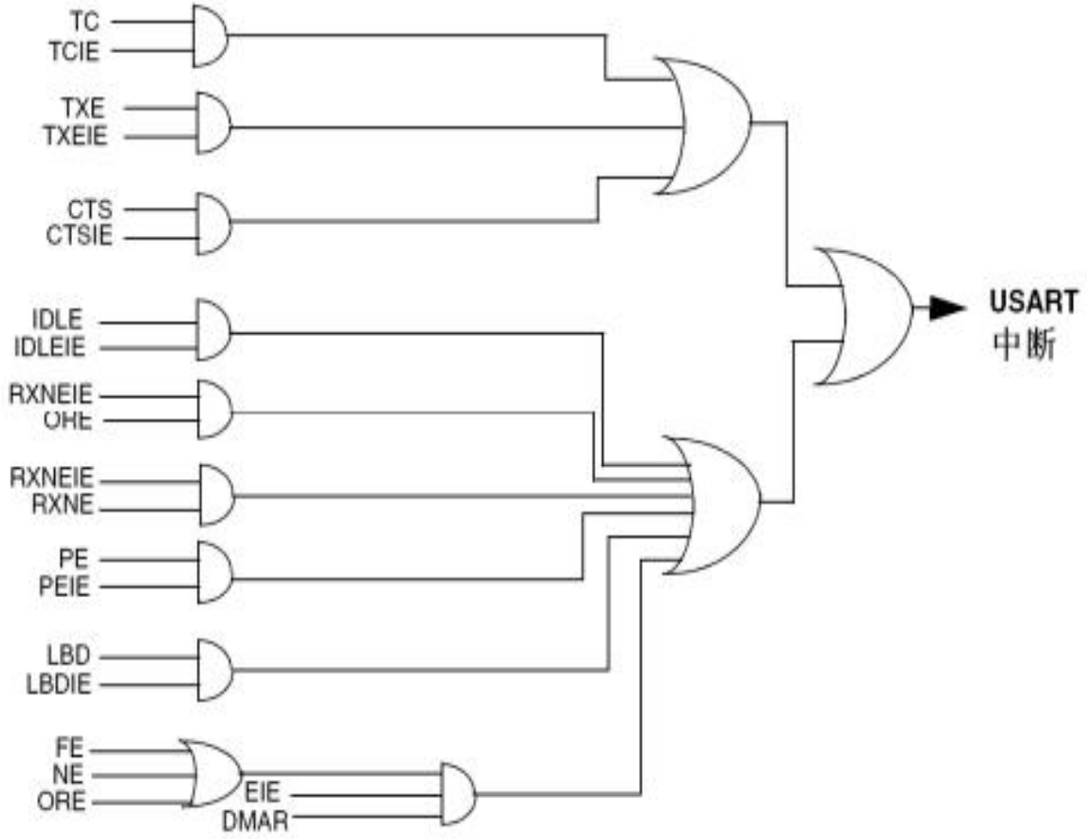
间接对寄存器操作实现相应的USART通信功能

7.3.3 USART编程模式

中断方式

USART主要有以下中断事件：

- 发送期间：发送完成、清除发送和发送数据寄存器空；
- 接收期间：空闲总线检测、溢出错误、接收数据寄存器非空、校验错误、LIN断开符号检测、噪音标志(仅在多缓冲器通信）和帧错误（仅在多缓冲器通信）。



USART中断映射

7.3.3 USART编程模式

中断方式

USART中断事件及中断使能标志位

中断标志	中断使能位	STM32中的定义	中断事件
TXE	TXEIE	UART_IT_TXE	发送数据寄存器空
CTS	CTSIE	UART_IT_CTS	CTS标志
TC	TCIE	UART_IT_TC	发送完成
RXNE	RXNEIE	UART_IT_RXNE	接收数据就绪(可读)
ORE	TXNEIE		检测到数据溢出
IDLE	IDLEIE	UART_IT_IDLE	检测到空闲线路标志
PE	PEIE	UART_IT_PE	奇偶检验错误标志
LBD	LBDIE	UART_IT_LBD	断开标志
NE或ORT或PE	EIE	UART_IT_ERR	噪声标志，溢出错误和帧错误

7.3.3 USART编程模式

DMA方式

USART模块可通过DMA方式进行大批量数据的连续传输。



STM32F103微控制器的USART的发送和接收分别映射到不同的DMA通道上，如USART1_Rx被映射到DMA1的通道5，USART1_Tx被映射到DMA1的通道4。

7.4 USART模块的标准外设库接口函数及应用

7.4.1 USART标准外设库接口函数

7.4.2 USART串口应用编程步骤

7.4.3 USART标准外设库应用实例

7.4.1 USART标准外设库接口函数

```
365 void USART_DeInit(USART_TypeDef* USARTx);
366 void USART_Init(USART_TypeDef* USARTx, USART_InitTypeDef* USART_InitStruct);
367 void USART_StructInit(USART_InitTypeDef* USART_InitStruct);
368 void USART_ClockInit(USART_TypeDef* USARTx, USART_ClockInitTypeDef* USART_ClockInitStruct);
369 void USART_ClockStructInit(USART_ClockInitTypeDef* USART_ClockInitStruct);
370 void USART_Cmd(USART_TypeDef* USARTx, FunctionalState NewState);
371 void USART_ITConfig(USART_TypeDef* USARTx, uint16_t USART_IT, FunctionalState NewState);
372 void USART_DMACmd(USART_TypeDef* USARTx, uint16_t USART_DMAReq, FunctionalState NewState);
373 void USART_SetAddress(USART_TypeDef* USARTx, uint8_t USART_Address);
374 void USART_WakeUpConfig(USART_TypeDef* USARTx, uint16_t USART_WakeUp);
375 void USART_ReceiverWakeUpCmd(USART_TypeDef* USARTx, FunctionalState NewState);
376 void USART_LINBreakDetectLengthConfig(USART_TypeDef* USARTx, uint16_t USART_LINBreakDetectLength);
377 void USART_LINCmd(USART_TypeDef* USARTx, FunctionalState NewState);
378 void USART_SendData(USART_TypeDef* USARTx, uint16_t Data);
379 uint16_t USART_ReceiveData(USART_TypeDef* USARTx);
380 void USART_SendBreak(USART_TypeDef* USARTx);
381 void USART_SetGuardTime(USART_TypeDef* USARTx, uint8_t USART_GuardTime);
382 void USART_SetPrescaler(USART_TypeDef* USARTx, uint8_t USART_Prescaler);
383 void USART_SmartCardCmd(USART_TypeDef* USARTx, FunctionalState NewState);
384 void USART_SmartCardNACKCmd(USART_TypeDef* USARTx, FunctionalState NewState);
385 void USART_HalfDuplexCmd(USART_TypeDef* USARTx, FunctionalState NewState);
386 void USART_OverSampling8Cmd(USART_TypeDef* USARTx, FunctionalState NewState);
387 void USART_OneBitMethodCmd(USART_TypeDef* USARTx, FunctionalState NewState);
388 void USART_IrDAConfig(USART_TypeDef* USARTx, uint16_t USART_IrDAMode);
389 void USART_IrDACmd(USART_TypeDef* USARTx, FunctionalState NewState);
390 FlagStatus USART_GetFlagStatus(USART_TypeDef* USARTx, uint16_t USART_FLAG);
391 void USART_ClearFlag(USART_TypeDef* USARTx, uint16_t USART_FLAG);
392 ITStatus USART_GetITStatus(USART_TypeDef* USARTx, uint16_t USART_IT);
393 void USART_ClearITPendingBit(USART_TypeDef* USARTx, uint16_t USART_IT);
```

STM32的USART标准外设库函数定义在STM32F10x标准外设库的stm32f10x_usart.c文件中，其相应的stm32f10x_usart.h头文件定义了USART相关的结构体和宏定义，并声明了相关的标准外设库函数，共29种。

7.4.1 USART标准外设库接口函数

USART初始化结构体

```
typedef struct
{
  uint32_t USART_BaudRate;
  uint16_t USART_WordLength;
  uint16_t USART_StopBits;
  uint16_t USART_Parity;
  uint16_t USART_Mode;
  uint16_t
USART_HardwareFlowControl;
} USART_InitTypeDef;
```

USART_InitTypeDef结构体成员及其取值

结构体成员	取值范围
USART_BaudRate (波特率)	9600 115200
USART_WordLength (数据位数)	USART_WordLength_8b(8位) USART_WordLength_9b(9位)
USART_StopBits (停止位)	USART_StopBits_1(1位) USART_StopBits_0_5(0.5位) USART_StopBits_2(2位) USART_StopBits_1_5(1.5位)
USART_Parity (校验模式)	USART_Parity_No(无校验) USART_Parity_Even(偶校验) USART_Parity_Odd(奇校验)
USART_Mode (收发模式)	USART_Mode_Rx(接收模式) USART_Mode_Tx(发送模式)
USART_HardwareFlow Control (硬件流控制)	USART_HardwareFlowControl_None (不使用联络信号) USART_HardwareFlowControl_RTS (使用RTS) USART_HardwareFlowControl_CTS (使用CTS) USART_HardwareFlowControl_RTS_ CTS(使用RTC和CTS)

7.4.1 USART标准外设库接口函数

库函数1

USART初始化函数

void USART_Init(USART_TypeDef* USARTx, USART_InitTypeDef* USART_InitStruct);

例如：

USART_Init(USART1,&USART_InitStructurer); //初始化串口1

库函数2

USART初始化复位函数

void USART_DeInit(USART_TypeDef* USARTx);

当外设出现异常时，可以通过复位操作，对外设重新进行配置。

例如：

USART_DeInit(USART1);//复位串口1

7.4.1 USART标准外设库接口函数

库函数3

USART使能函数

void USART_Cmd(USART_TypeDef* USARTx, FunctionalState NewState);

例如：

USART_Cmd(USART1,ENABLE);//使能串口1

库函数4

USART发送单个数据函数

void USART_SendData(USART_TypeDef* USARTx, uint16_t Data);

例如：

USART_SendData(USART1,0x26)；//通过串口1发送十六进制数据26H

7.4.1 USART标准外设库接口函数

库函数5

USART接收数据函数

uint16_t USART_ReceiveData(USART_TypeDef* USARTx);

例如：

Temp = USART_SendData(USART1); //将接收串口1发来的数据存到变量Temp中

库函数6

USART获取串口状态函数

FlagStatus USART_GetFlagStatus(USART_TypeDef* USARTx, uint16_t USART_FLAG);

使用串口进行数据发送或接收时，先判断状态寄存器的状态再进行下一步操作，该函数实现对指定的USART标志位是否设置进行检查。

例如：FlagStatus Status;

Status = USART_GetFlagStatus(USART1,USART_FLAG_TXE);

7.4.1 USART标准外设库接口函数

库函数6

USART获取串口状态函数

函数USART_GetFlagStatus()有两个输入参数：

- 参数1：USARTx，可选为USART1、USART2或USART3；
- 参数2：USART_FLAG为待检测的USART标志位；
- 函数返回值为SET(置位)或RESET(复位）。

USART_FLAG	描述
USART_FLAG_CTS	CTS标志位
USART_FLAG_LBD	LIN中断检测标志位
USART_FLAG_TXE	发送数据寄存器空标志位
USART_FLAG_TC	发送完成标志位
USART_FLAG_RXNE	接收数据寄存器非空标志位
USART_FLAG_IDLE	空闲总线标志位
USART_FLAG_ORE	溢出错误标志位
USART_FLAG_NE	噪声错误标志位
USART_FLAG_FE	帧错误标志位
USART_FLAG_PE	奇偶错误标志位

7.4.1 USART标准外设库接口函数

USART获取串口状态函数

库函数6

串口采用USART_SendData()函数发送数据时，要先判断USART_FLAG_TXE(发送数据寄存器空标志位）或USART_FLAG_TC(发送完成标志位）的状态，等发送完成后再执行后面的程序，具体实现代码如下:

```
USART_SendData(USART1，(uint8_t) ch);  //发送数据
//判断发送是否完成，循环等待直到发送完毕
while(USART_GetFlagStatus(USART1,USART_FLAG_TC) != SET);
return ch;
```

或

```
while (USART_GetFlagStatus(DEBUG_USARTx, USART_FLAG_TXE)
== RESET);
return (ch);
```

7.4.1 USART标准外设库接口函数

库函数6

USART获取串口状态函数

采用USART_ReceiveData()函数进行接收数据时，要先判断USART_FLAG_RXNE(接收数据寄存器非空标志位），非空时接收数据，该位被置“1”时，表示有数据被接收，并可以读出。具体代码如下:

```
    //判断接收寄存器是否为空，非空接收数据
if(USART_GetFlagStatus(USART1,USART_FLAG_RXNE) == SET)
{
    ch = USART_ReceiveData(USART1); //接收数据
}
```

7.4.1 USART标准外设库接口函数

USART清除状态标志函数

库函数7

void USART_ClearFlag(USART_TypeDef* USARTx, uint16_t USART_FLAG);

例如：

USART_ClearFlag(USART1,USART_FLAG_TC);//清除串口1的TC状态标志位

USART串口中断使能函数

库函数8

void USART_ITConfig(USART_TypeDef* USARTx, uint16_t USART_IT, FunctionalState NewState);

例如：

USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
　　//开启接收数据寄存器非空中断

USART_ITConfig(USART1, USART_IT_TC, ENABLE); //开启发送完成中断

7.4.1 USART标准外设库接口函数

库函数8

USART串口中断使能函数

USART_ITConfig()函数的参数：

- 参数1：USARTx，可选为USART1、USART2或USART3；
- 参数2：USART_IT为待检测的USART标志位；
- 参数3为ENABLE或DISABLE；
- 函数返回值为SET(置位)或RESET(复位）。

USART_IT	描述
USART_IT_PE	奇偶错误中断
USART_IT_TXE	发送中断
USART_IT_TC	传输完成中断
USART_IT_RXNE	接收中断
USART_IT_IDLE	空闲总线中断
USART_IT_LBD	LIN中断检测中断
USART_IT_CTS	CTS中断
USART_IT_ERR	错误中断

7.4.1 USART标准外设库接口函数

库函数9

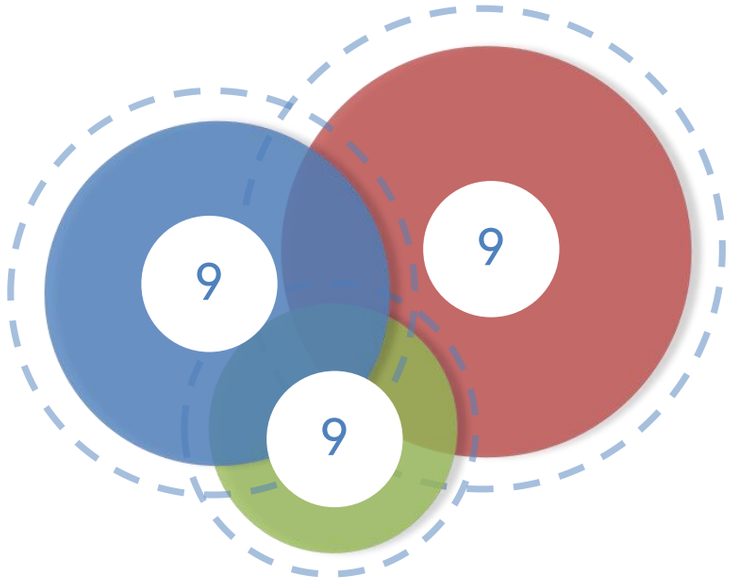
USART获取串口中断状态函数

ITStatus USART_GetITStatus(USART_TypeDef* USARTx, uint16_t USART_IT);

若使能了中断，则会设置状态寄存器中的某个标志位，通过调用该函数来判断是否是串口发送完成中断，实现方式如下：

USART_GetITStatus(USART1, USART_IT_TC);

若返回值是SET，则说明串口1发送完成，产生了中断。



7.4.1 USART标准外设库接口函数

库函数10

USART清除串口中断标志函数

void USART_ClearITPendingBit(USART_TypeDef* USARTx, uint16_t USART_IT);

例如，复位发送完成中断标志位：

USART_ClearITPendingBit(USART1,USART_IT_TC);

库函数11

USART串口DMA使能函数

void USART_DMACmd(USART_TypeDef* USARTx, uint16_t USART_DMAReq, FunctionalState NewState);

用于配置DMA与USART通道，例如：

```
USART_DMACmd(USART1, USART_DMAReq_Tx, ENABLE);
    //使能USART1的发送DMA请求
USART_DMACmd(USART1, USART_DMAReq_RX, ENABLE);
    //使能USART1的接收DMA请求
```

7.4.2 USART串口应用编程步骤



7.4.2 USART串口应用编程步骤

1

声明GPIO和USART初始化结构体

USART_InitTypeDef USART_InitStructure;

2

串口所用的GPIO时钟使能，串口时钟使能

开启外设时钟RCC_APB2PeriphClockCmd()。例如：使能USART1、GPIOA的时钟，所用的函数为

RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|RCC_APB2Periph_GPIOA, ENABLE);

7.4.2 USART串口应用编程步骤

3

设置I/O引脚功能为复用推挽输出、浮空输入

- 串口使用的是I/O的复用功能。USART1的发送引脚为PA9，需将PA9配置为复用推挽输出；
- USART1的输入引脚为PA10，需将PA10配置为浮空输入。

```
    //USART1_TX，PA9，配置为复用推挽输出，并初始化PA9
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    //复用推挽输出
GPIO_Init(GPIOA, &GPIO_InitStructure);
    //USART1_RX，PA10，配置为浮空输入，并初始化PA10
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING; //浮空输入
```

7.4.2 USART串口应用编程步骤

4

设置波特率，设置数据格式：数据位、停止位、校验位

```
USART_InitStructure.USART_BaudRate = 115200; //串口波特率
USART_InitStructure.USART_WordLength= USART_WordLength_8b;
  //设置数据位占8位
USART_InitStructure.USART_StopBits = USART_StopBits_1; //设置停止位占1位
USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
  //不采用硬件数据流控制
USART_InitStructure.USART_HardwareFlowControl=USART_HardwareFlowC
ontrol_None;
USART_InitStructure.USART_Mode=USART_Mode_Rx|USART_Mode_Tx;
  //设置为收发模式
```

7.4.2 USART串口应用编程步骤

5

使用串口初始化函数USART_Init()初始化相应串口

USART_Init(USART1, &USART_InitStructure);//初始化串口1

USART1

6

利用串口使能函数USART_Cmd()使能相应串口

USART_Cmd(USART1, ENABLE);//使能串口

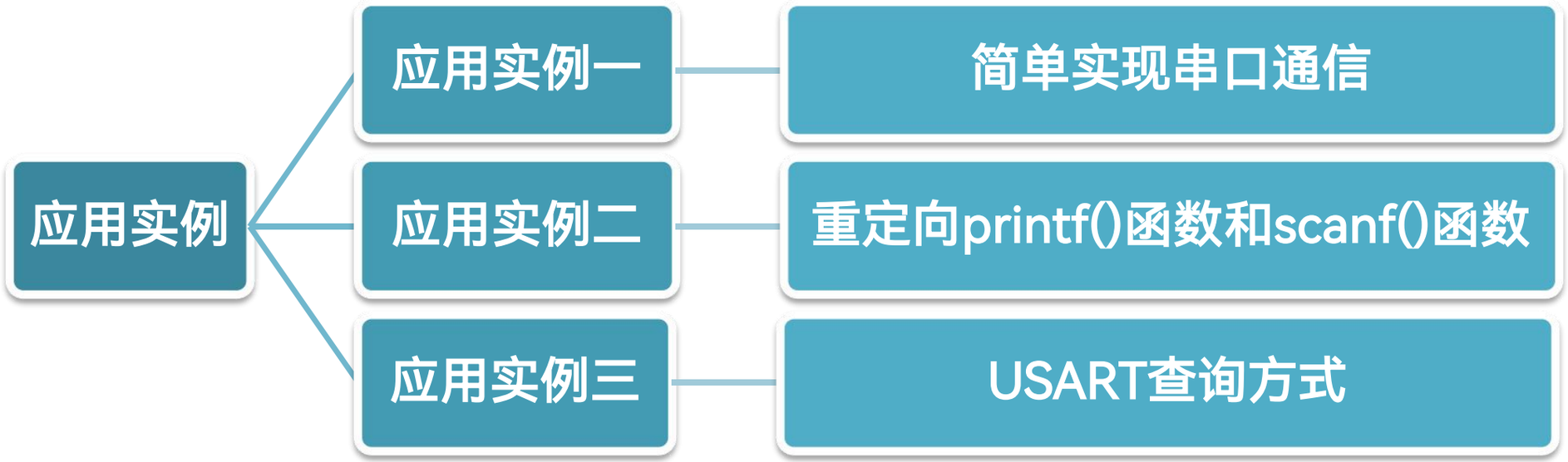
7

应用程序编写

若使用中断，则编写串口中断函数void

USART1_IRQHandler(void)

7.4.3 USART标准外设库应用实例



7.4.3 USART标准外设库应用实例

应用实例———简单实现串口通信

编程代码

USART初始化头文件USART_Init_Config.h

USART初始化程序USART_Init_Config.c文件

主程序main.c文件

功能描述：

通过串口USART1实现简单的收发功能。

通过串口调试助手进行数据的收发显示。

USART初始化头文件USART_Init_Config.h，代码如下：

```
#ifndef __USART_INIT_CONFIG_H
#define __USART_INIT_CONFIG_H

#include "stm32f10x.h"
void USART_Init_Config(void);

#endif
```

7.4.3 USART标准外设库应用实例

应用实例——简单实现串口通信

USART初始化程序USART_Init_Config.c文件

```
#include <stdio.h>
#include "USART_Init_Config.h"

void USART_Init_Config(void)
{
GPIO_InitTypeDef GPIO_InitStructure;
USART_InitTypeDef USART_InitStructure;
   //开启GPIOA和USART1的时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|RCC_APB2Periph_GPIOA, ENABLE);
   //USART1_TX，设置PA9为复用输出模式
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9; //PA9
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
GPIO_Init(GPIOA, &GPIO_InitStructure);
   //USART1_RX，设置PA10为浮空输入模式
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIOA, &GPIO_InitStructure);
  //USART1参数配置
USART_InitStructure.USART_BaudRate =
115200;
   //设置波特率为115200
USART_InitStructure.USART_WordLength =
USART_WordLength_8b; //数据位占8位
USART_InitStructure.USART_StopBits =
USART_StopBits_1; //1位停止位
USART_InitStructure.USART_Parity =
USART_Parity_No; //无校验
USART_InitStructure.USART_HardwareFlowCon
trol=USART_HardwareFlowControl_None;
USART_InitStructure.USART_Mode =
USART_Mode_Rx | USART_Mode_Tx;
USART_Init(USART1, &USART_InitStructure);
    //初始化串口1
USART_Cmd(USART1, ENABLE); //使能串口1
}
```

7.4.3 USART标准外设库应用实例

应用实例——简单实现串口通信

主程序main.c文件，代码如下：

```
#include "stm32f10x.h"
#include <stdio.h>
#include "led.h"
#include "USART_Init_Config.h"

int main(void)
{  LED_Init();
   USART_Init_Config();
   char Temp;
   while(1)
   {

if(USART_GetFlagStatus(USART1,USART_FLAG_RX
NE))
     {  USART_ClearFlag(USART1,USART_FLAG_TC);
        Temp = USART_ReceiveData(USART1);
        USART_SendData(USART1,Temp);

USART_ClearFlag(USART1,USART_FLAG_RXNE);
     }
```

另一种实现方式：

```
int main(void)
{
   while(1)
   {

       if(USART_GetFlagStatus(USART1,USART_I
       T_RXNE) == SET)
    {
         ch = USART_ReceiveData(USART1); //接
收数据
        USART_SendData(USART1,ch); //发送数
据
            //循环等待发送完毕
        while(USART_GetFlagStatus(USART1,
       USART_FLAG_TC) != SET);
     }
   }
}
```

7.4.3 USART标准外设库应用实例

应用实例二——重定向printf()函数和scanf()函数

- 在嵌入式C语言开发中需要重新定义printf()函数和scanf()函数，将标准的输入和输出重新定向到串口，才能实现串口输出或LCD输出。
- 重定向printf()函数和scanf()函数也就是重写fput()函数和fget()函数。

	fputc()函数	fgetc()函数
函数原型	int fputc(int ch,FILE *f)	int fgetc(FILE *f)
功能	将一个字符写入到文件中	从文件f中读出一个字符
参数	ch为要输出的字符，f指向FILE结构的指针	f指向FILE结构的指针
返回值	发送成功，返回ch的值；失败，则返回EOF(通常为-1)	成功，则返回读取的字符；失败，则返回EOF(通常为-1)

7.4.3 USART标准外设库应用实例

应用实例二——重定向printf()函数和scanf()函数

- 重定向printf()函数和scanf()函数，需要在串口初始化头文件usart.h中添加#include <stdio.h>头文件，同时在Keil MDK编译器的Target中勾选Use MicroLIB。
- 按照fget()函数和fput()函数的原型，重写相应的USART接口程序

```
//重定向C库函数printf()到串口
int fputc(int ch, FILE *f)
{
        /* 发送一个字节数据到串口 */
  USART_SendData(DEBUG_USARTx, (uint8_t) ch);
        /*等待发送完毕 */
  while (USART_GetFlagStatus(DEBUG_USARTx, USART_FLAG_TXE) == RESET);
        return (ch);
}
```

7.4.3 USART标准外设库应用实例

应用实例二——重定向printf()函数和scanf()函数

- 使用上述代码输出字符串时，会出现丢失第一个字符的现象，可以在发送字符前调用USART_ClearFlag()复位TC标志位

```
USART_ClearFlag(USARTx,USART_FLAG_TC);
    //重定向C库函数scanf()到串口，重定向后可使用scanf()函数
int fgetc(FILE *f)
{     /*等待串口输入数据  */
    while (USART_GetFlagStatus(DEBUG_USARTx,
USART_FLAG_RXNE)==RESET);
            return (int)USART_ReceiveData(DEBUG_USARTx);
}
```

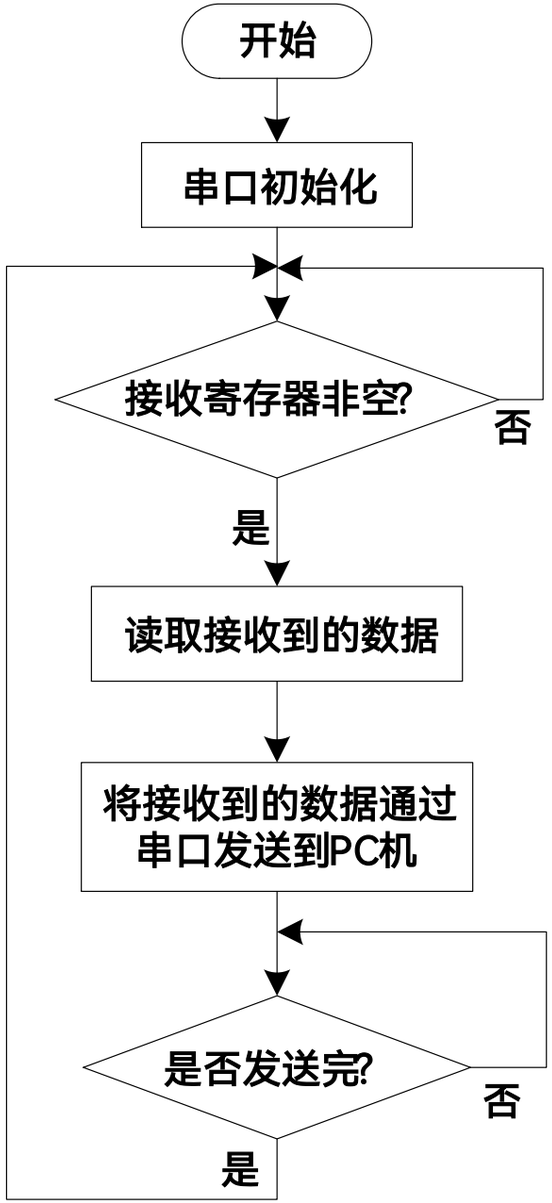
7.4.3 USART标准外设库应用实例

应用实例三——USART查询方式

功能描述

通过串口1利用查询方式实现发送字符命令“Y”点亮LED灯，发送字符命令“N”熄灭LED灯。

USART可以采用**查询方式**进行数据通信，根据各状态寄存器的状态决定下一步是否要执行发送接收或转入相应错误处理。



7.4.3 USART标准外设库应用实例

应用实例三——USART查询方式



USART初始化头文件USART_Init_Config.h，代码如下：

```
#ifndef __USART_INIT_CONFIG_H
#define __USART_INIT_CONFIG_H

#include "stm32f10x.h"
void USART_Init_Config(void);

#endif
```

7.4.3 USART标准外设库应用实例

应用实例三——USART查询方式

USART初始化程序USART_Init_Config.c文件，代码如下：

```
#include <stdio.h>
#include "USART_Init_Config.h"

void USART_Init_Config(void)
{
  GPIO_InitTypeDef GPIO_InitStructure;
  USART_InitTypeDef USART_InitStructure;
    //使能USART1和GPIOA的时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|RCC_APB2Periph_GPIOA, ENABLE);
  USART_DeInit(USART1);  //复位串口1
     //USART1_TX   PA9为复用推挽输出
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9; //PA.9
  GPIO_InitStructure.GPIO_Speed =
GPIO_Speed_50MHz;
  GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_AF_PP;    GPIO_Init(GPIOA,
&GPIO_InitStructure);
     //USART1_RX   PA10为浮空输入
GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIOA, &GPIO_InitStructure);
//USART1参数配置：波特率115200，8位数据位，1位停止位，无校验，不使用硬件流控制
USART_InitStructure.USART_BaudRate = 115200;
USART_InitStructure.USART_WordLength =
USART_WordLength_8b;
USART_InitStructure.USART_StopBits =
USART_StopBits_1;
USART_InitStructure.USART_Parity =
USART_Parity_No;
USART_InitStructure.USART_HardwareFlowControl
=SART_HardwareFlowControl_None;
USART_InitStructure.USART_Mode =
USART_Mode_Rx | USART_Mode_Tx;
USART_Init(USART1, &USART_InitStructure);
USART_Cmd(USART1, ENABLE);
}
```

7.4.3 USART标准外设库应用实例

应用实例三——USART查询方式

```
    //重定向printf函数
int fputc(int ch, FILE *f)
{
      /* 发送一个字节数据到USART1*/
   USART_SendData(USART1, (uint8_t)
ch);
      /*等待发送寄存器为空 */
   while
  (USART_GetFlagStatus(USART1,
  USART_FLAG_TXE) == RESET);

        return (ch);
}
```

```
    //重定向scanf函数
int fgetc(FILE *f)
{
     /*等待串口1输入数据 */
    while
   (USART_GetFlagStatus(USART1,
   USART_FLAG_RXNE) == RESET);
       return
      (int)USART_ReceiveData(USART
      1);
}
```

7.4.3 USART标准外设库应用实例

应用实例三——USART查询方式

main.c程序

```
#include "stm32f10x.h"
#include <stdio.h>
#include "led.h"
#include "USART_Init_Config.h"

int main(void)
{
    char ch;
     //定义一个字符变量，用于接收字符
   LED_Init(); //LED灯初始化
   USART_Init_Config(); //USART初始化
   while(1)
   {
      ch = getchar();
       printf("接收到的指令 %c \n",ch);
       switch(ch)
       {
          case 'N':
             GPIO_SetBits(GPIOB,GPIO_Pin_5);
             printf("熄灭LED灯 \n");
          break;
          case 'Y':
              GPIO_ResetBits(GPIOB,GPIO_Pin_5);
              printf("点亮LED灯 \n");
          break;
          default:
               printf("请输入正确的指令：N熄灭灯，
              Y点亮灯!\n");
          break;
        }
    }
}
```

7.5 USART模块的HAL库接口函数及应用



7.5.1 UART的HAL库接口函数

7.5.2 USART的HAL库应用实例

7.5.3 USART模块应用实例拓展

7.5.1 UART的HAL库接口函数

```
/* Initialization and de-initialization functions  ****************************/
HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart);
HAL_StatusTypeDef HAL_HalfDuplex_Init(UART_HandleTypeDef *huart);
HAL_StatusTypeDef HAL_LIN_Init(UART_HandleTypeDef *huart, uint32_t BreakDetectLength);
HAL_StatusTypeDef HAL_MultiProcessor_Init(UART_HandleTypeDef *huart, uint8_t Address, uint32_t WakeUpMethod);
HAL_StatusTypeDef HAL_UART_DeInit (UART_HandleTypeDef *huart);
void HAL_UART_MspInit(UART_HandleTypeDef *huart);
void HAL_UART_MspDeInit(UART_HandleTypeDef *huart);

/**
  * @}
  */

/** @addtogroup UART_Exported_Functions_Group2 IO operation functions
  * @{
  */

/* IO operation functions *****************************************************/
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
HAL_StatusTypeDef HAL_UART_DMAPause(UART_HandleTypeDef *huart);
HAL_StatusTypeDef HAL_UART_DMAResume(UART_HandleTypeDef *huart);
HAL_StatusTypeDef HAL_UART_DMAStop(UART_HandleTypeDef *huart);
void HAL_UART_IRQHandler(UART_HandleTypeDef *huart);
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart);
void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart);
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart);
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart);
```

UART模块的HAL库相关接口函数定义在源文件stm32f1xx_hal_uart.c中，在stm32f1xx_hal_uart.h头文件中可以查看相关库函数声明以及相关的结构体定义。

7.5.1 UART的HAL库接口函数

USART模块的HAL库常用接口函数

类型		函数及功能描述
引脚功能操作函数	轮询方式	HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout); 功能描述：串口轮询模式发送函数，使用超时管理机制
		HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout); 功能描述：串口轮询模式接收函数，使用超时管理机制
	中断方式	HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口中断模式发送函数
		HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口中断模式接收函数
	DMA方式	HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口DMA模式发送函数
		HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口DMA模式接收函数
		HAL_UART_DMAPause(UART_HandleTypeDef *huart); 功能描述：串口DMA模式暂停函数
		HAL_UART_DMAStop(UART_HandleTypeDef *huart); 功能描述：串口DMA模式停止函数

7.5.1 UART的HAL库接口函数

USART模块的HAL库常用接口函数

类型	函数及功能描述
初始化及复位函数	HAL_UART_Init(UART_HandleTypeDef *huart) 功能描述：UART初始化函数
	HAL_UART_DeInit(UART_HandleTypeDef *huart) 功能描述：UART复位函数
中断处理函数(回调函数)	void HAL_UART_IRQHandler(UART_HandleTypeDef *huart); 功能描述：UART中断处理函数
	void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据发送完成时的回调函数，用户在该函数内编写中断处理程序
	void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据发送一半时的回调函数
	void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据接收完成时的回调函数，用户在该函数内编写中断处理程序
	void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据接收一半时的回调函数
	void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart); 功能描述：UART传输出现错误时调用的回调函数
外设状态函数	HAL_UART_GetState(UART_HandleTypeDef *huart); 功能描述：获取串口状态函数
	HAL_UART_GetError(UART_HandleTypeDef *huart); 功能描述：获取串口错误函数

7.5.1 UART的HAL库接口函数

UART发送函数

HAL_UART_Transmit()

函数原型

```
HAL_StatusTypeDef  HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

函数解析

该函数为串口在轮询方式下发送指定长度的数据，采用超时管理机制，若超出设定值时数据还没有发送完成，则不再发送，返回超时标志（HAL_TIMEOUT）。

使用范例

采用串口2发送固定数据，无限等待，间隔1s循环发送。

```
const char  *Send_Data = "Welcome to School !\r\n";
HAL_UART_Transmit(&huart2, (uint8_t *)
    Send_Data, strlen(Send_Data), 0xFFFF);
HAL_Delay(1000);
```

7.5.1 UART的HAL库接口函数

UART发送函数

HAL_UART_Transmit()

HAL_UART_Transmit()共有四个参数



第一个参数huart为结构体类型UART_HandleTypeDef定义的指针，取值为huart1 ~ huart5。



第二个参数uint8_t *pData为要发送的数据，pData为指向数据缓冲区的指针，类型为uint8_t，若发送的数据不是此类型，需要进行类型转换。



第三个参数uint16_t Size为要发送数据的大小。



第四个参数uint32_t Timeout为超时等待时间。

7.5.1 UART的HAL库接口函数

UART接收函数

HAL_UART_Receive()

函数原型

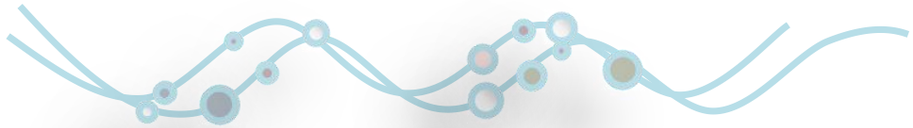
```
HAL_StatusTypeDef  HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

函数解析

HAL_UART_Receive()为UART在轮询模式下的接收函数，采用超时管理机制，使用方式同HAL_UART_Transmit()

使用范例

```
uint8_t  ch;
HAL_UART_Receive(&huart3,(uint8_t *)&ch, 1, 1000);
```



7.5.1 UART的HAL库接口函数

```
HAL_StatusTypeDef
HAL_UART_Receive_IT(UART_HandleTypeDef
*huart, uint8_t *pData, uint16_t Size)
{
 if (huart->RxState == HAL_UART_STATE_READY)
     //如果串口处于空闲状态，则执行以下语句
 {
     if ((pData == NULL) || (Size == 0U))
  {      return HAL_ERROR;
  }
   __HAL_LOCK(huart); //配置huart参数前先上锁
      /*以下为结构体变量huart的参数配置              */
   huart->pRxBuffPtr = pData;  //设置缓存指针
   huart->RxXferSize = Size;  //设置接收数据数量
   huart->RxXferCount = Size; //设置接收计数器
   huart->ErrorCode = HAL_UART_ERROR_NONE;
      //ErrorCode设置为无错误
   huart->RxState = HAL_UART_STATE_BUSY_RX;
     //接收状态设置为接收忙碌
```

UART中断接收函数HAL_UART_Receive_IT( )

```
      /* huart解锁 */
   __HAL_UNLOCK(huart);
      /*使能UART的PE(奇偶检验错误) 中断 */
   __HAL_UART_ENABLE_IT(huart, UART_IT_PE);
     /*使能UART错误中断*/
   __HAL_UART_ENABLE_IT(huart, UART_IT_ERR);
      /*使能UART数据寄存器非空中断*/
   __HAL_UART_ENABLE_IT(huart, UART_IT_RXNE);
   return HAL_OK; //配置完成，则返回HAL_OK
 }
 else
 {    return HAL_BUSY;
     //若huart不是READY状态，则返回HAL_BUSY忙
状态
 }
}
```

例： uint8_t RxBuffer[10];
HAL_UART_Receive_IT(&huart1,(uint8_t *)&RxBuffer,1);

7.5.2 USART的HAL库应用实例

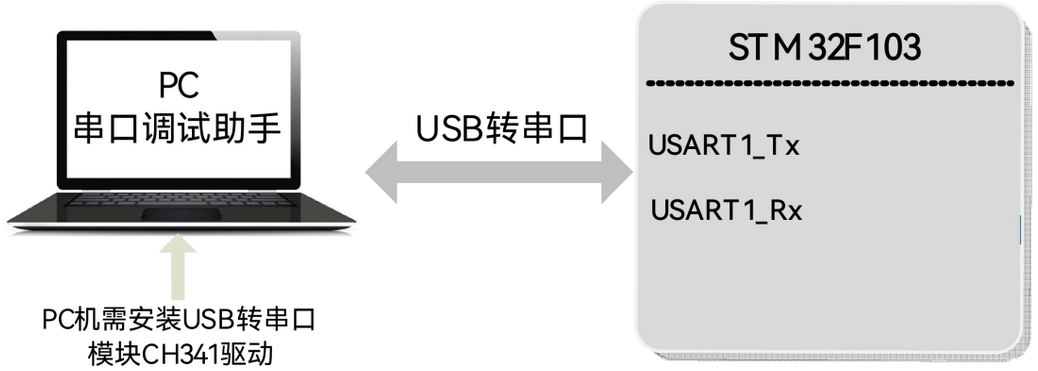
功能

实现简单的STM32串口发送和接收数据，调试此应用实例，需要在PC机端安装“串口调试助手”。

实例效果：

STM32目标板每隔1s循环发送字符“欢迎来到串口通信模块”到PC机上的串口助手。

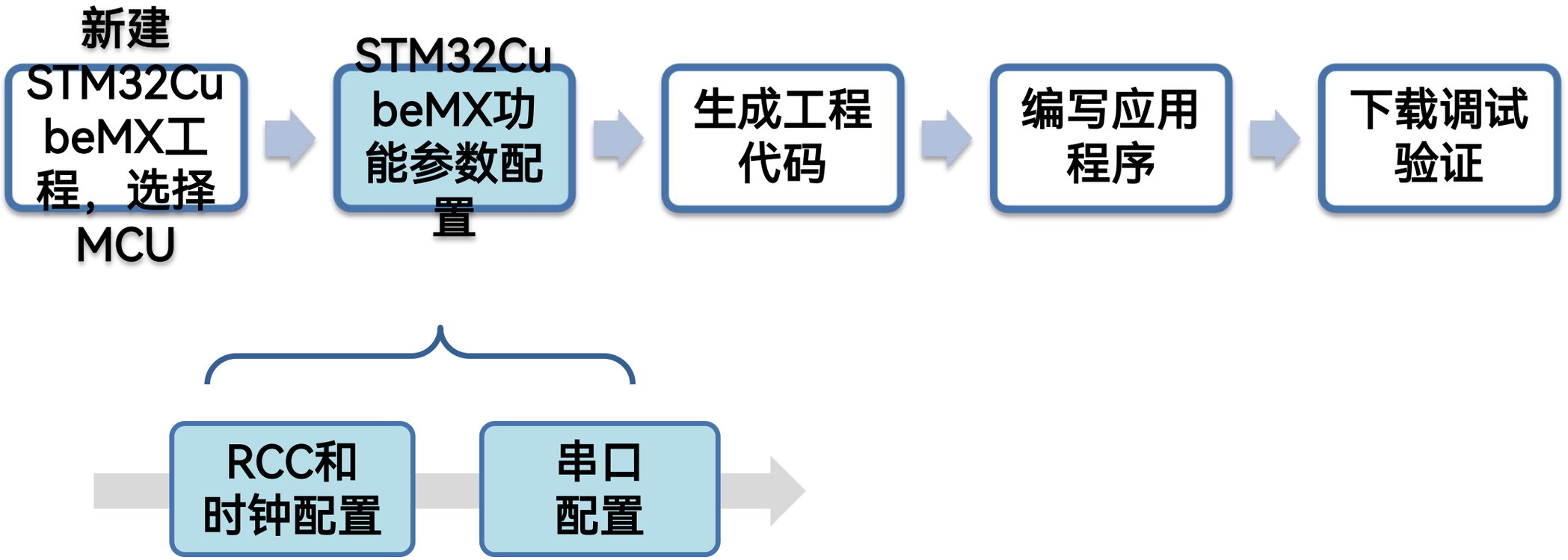
硬件设计



通过STM32F103目标板上的串口1实现与上位机的通信，可通过串口调试助手查看测试结果。

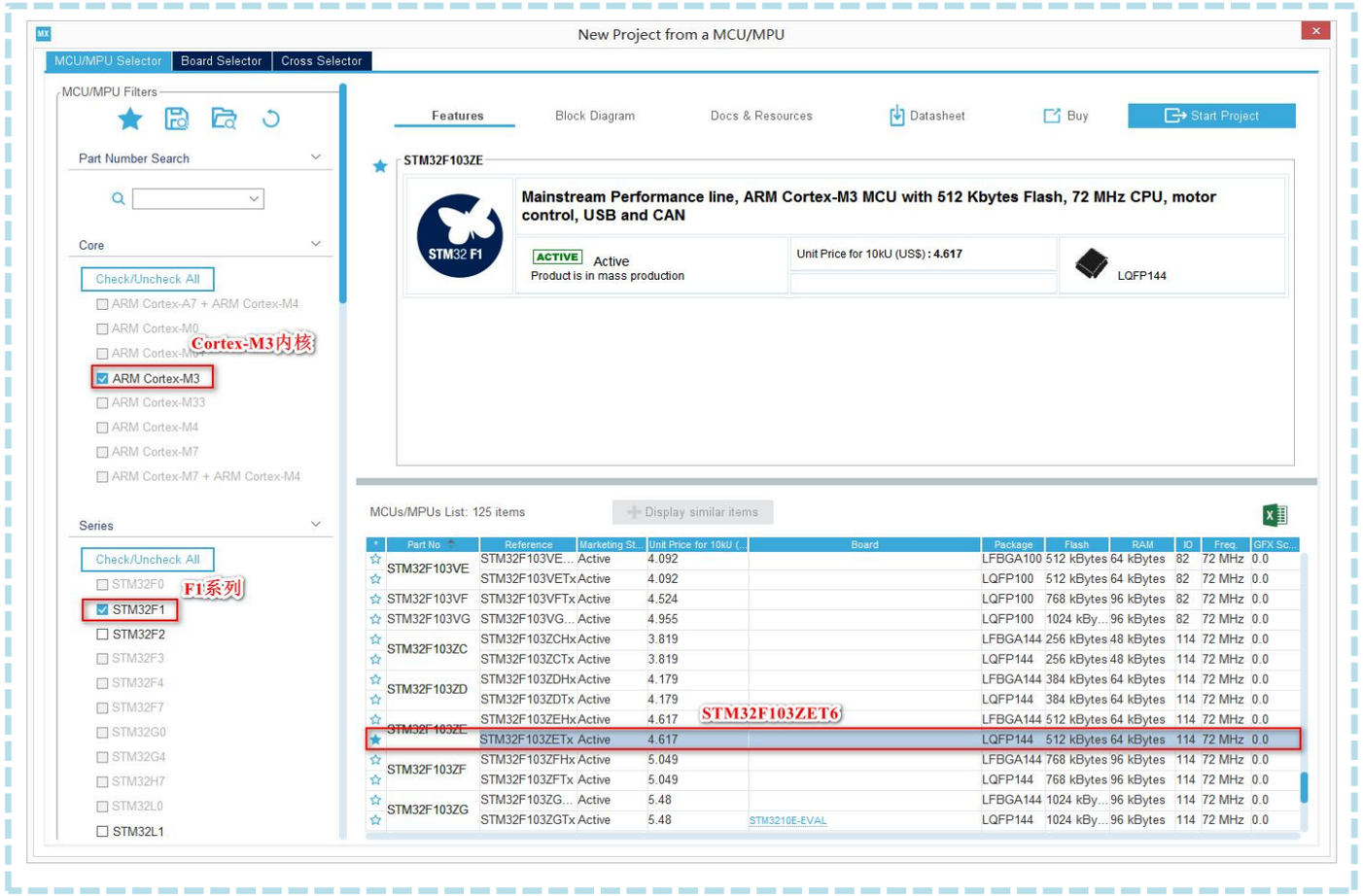
7.5.2 USART的HAL库应用实例

软件设计流程



软件设计——新建STM32CubeMX工程，选择MCU

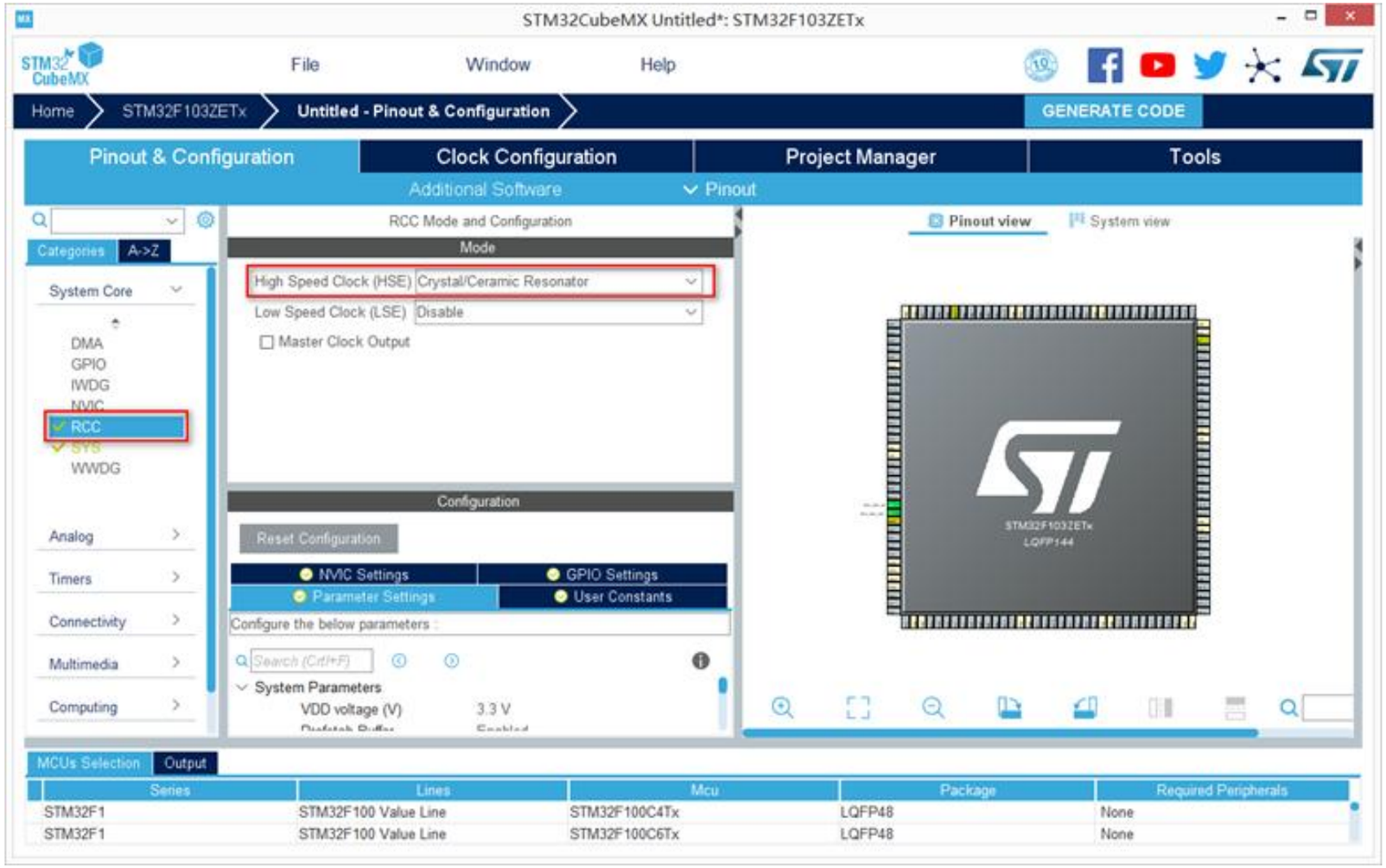
新建一个STM32CubeMx工程，选择选择MCU，这里选择STM32F103ZETx芯片，读者可根据自己的目标板选择相应的芯片



软件设计——STM32CubeMX功能参数配置

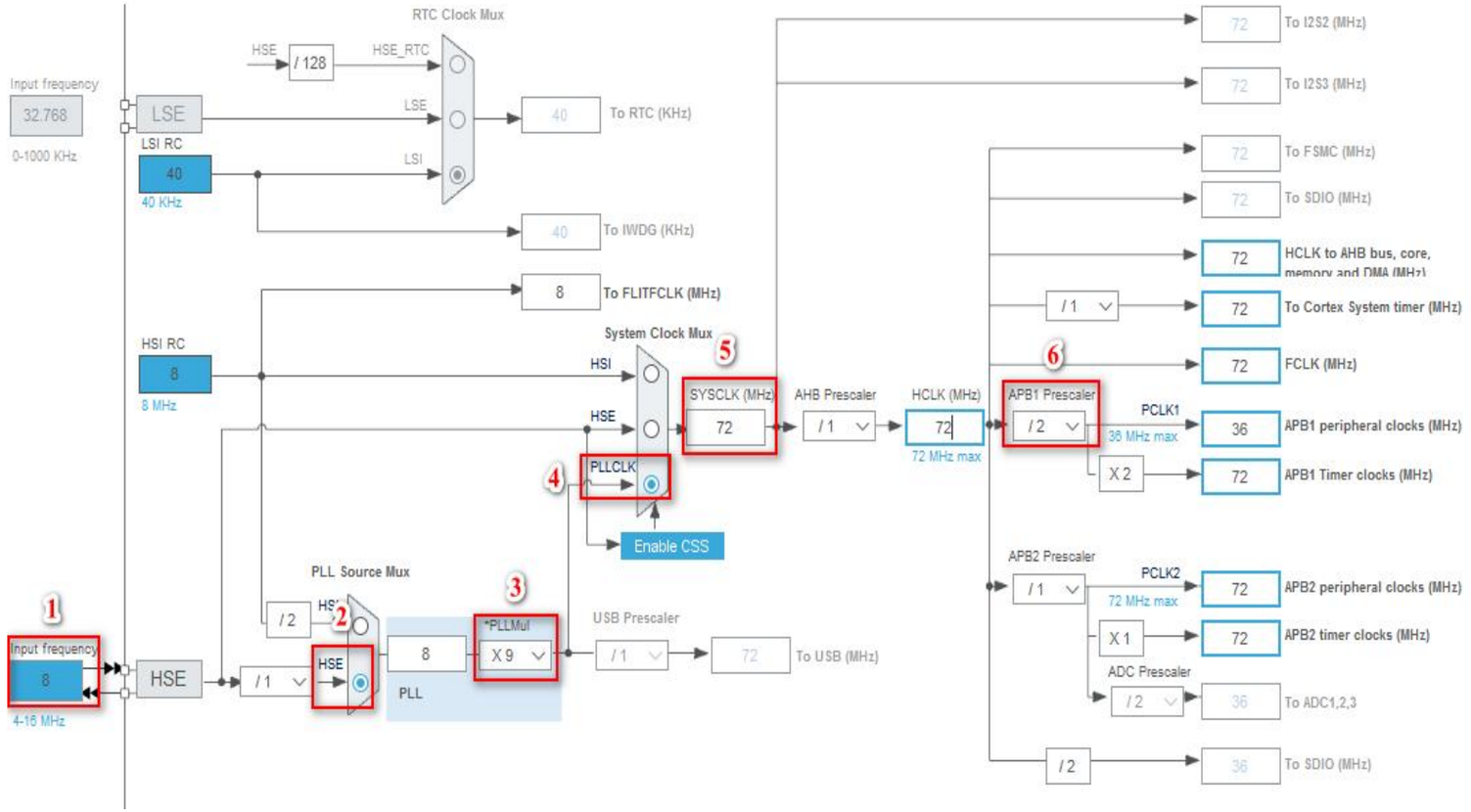
RCC配置

HSE选择“Crystal/Ceramic Resonator”(晶振/陶瓷谐振器



软件设计——STM32CubeMX功能参数配置

时钟配置

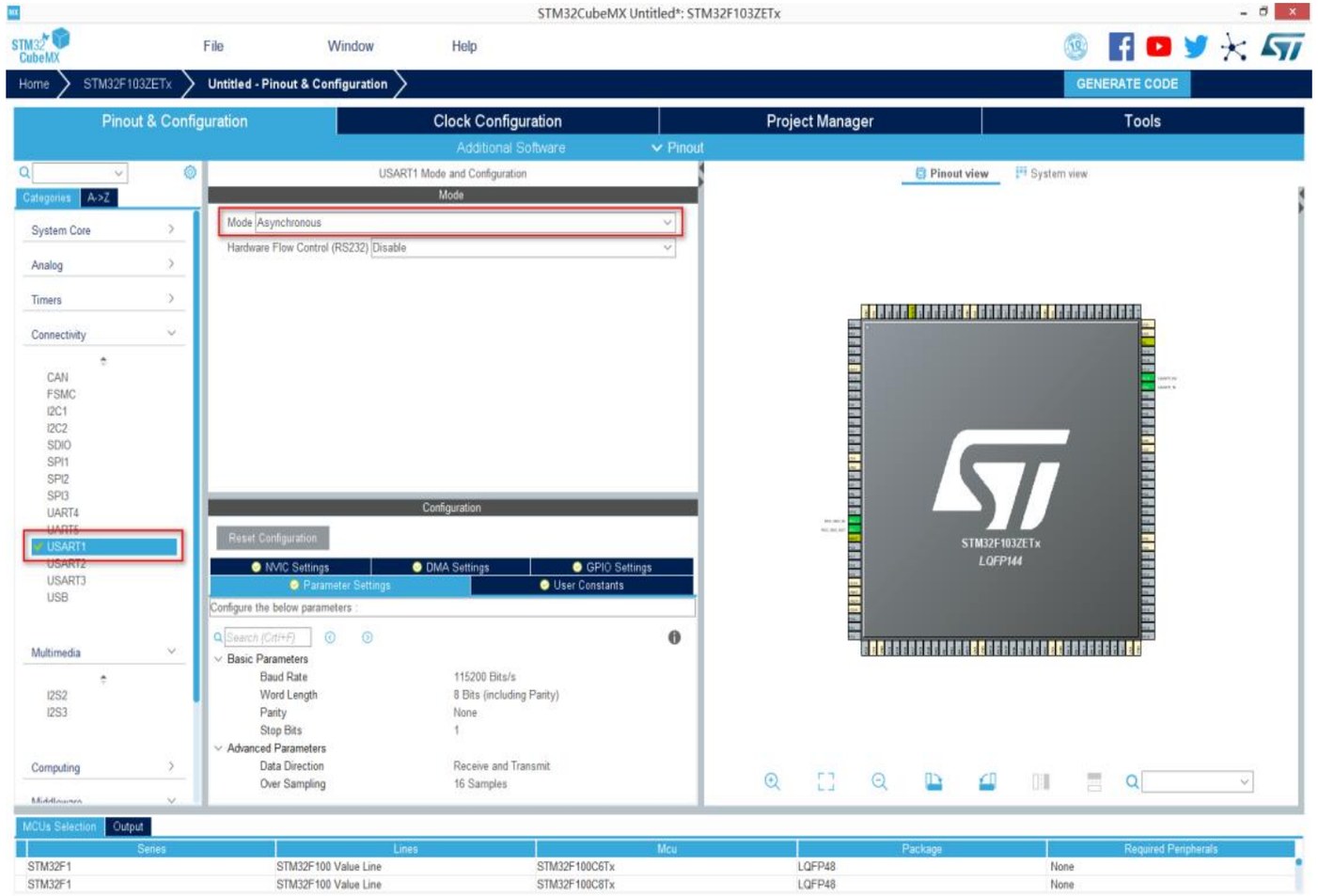


通过图形化方式直观地对系统时钟进行配置，系统时钟采用外部高速时钟，配置STM32F103系列芯片最大时钟为72MHz，配置APB2为72MHz，配置APB1为36MHz。

软件设计——STM32CubeMX功能参数配置

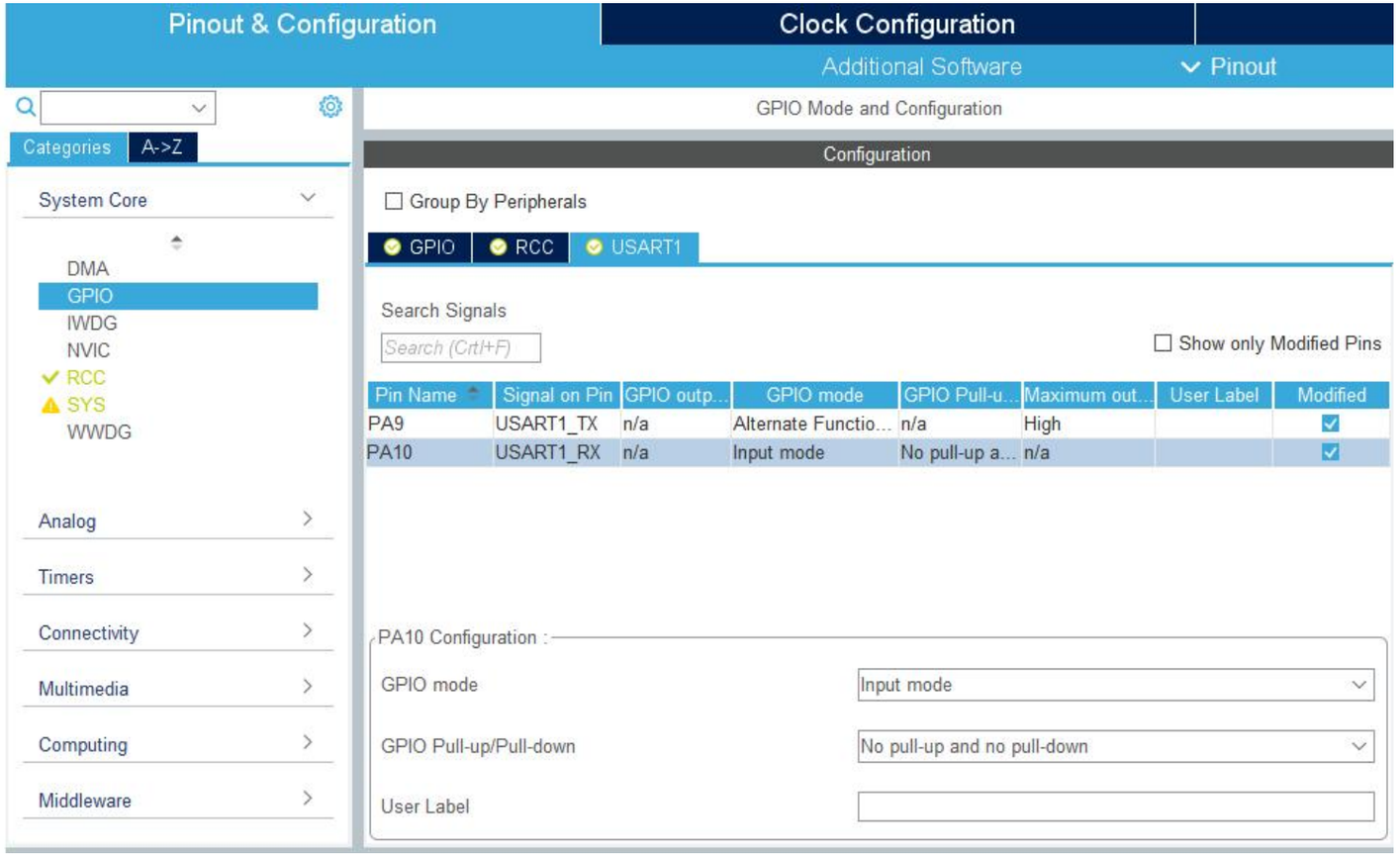
串口配置

- 在“Connectivity”中选择“USART1”；
- 设置“Mode”为“Asynchronous”(异步传输模式)
- 设置USART1的相关参数，配置为默认值：115200、8、None和1。



软件设计——STM32CubeMX功能参数配置

串口引脚参数设置

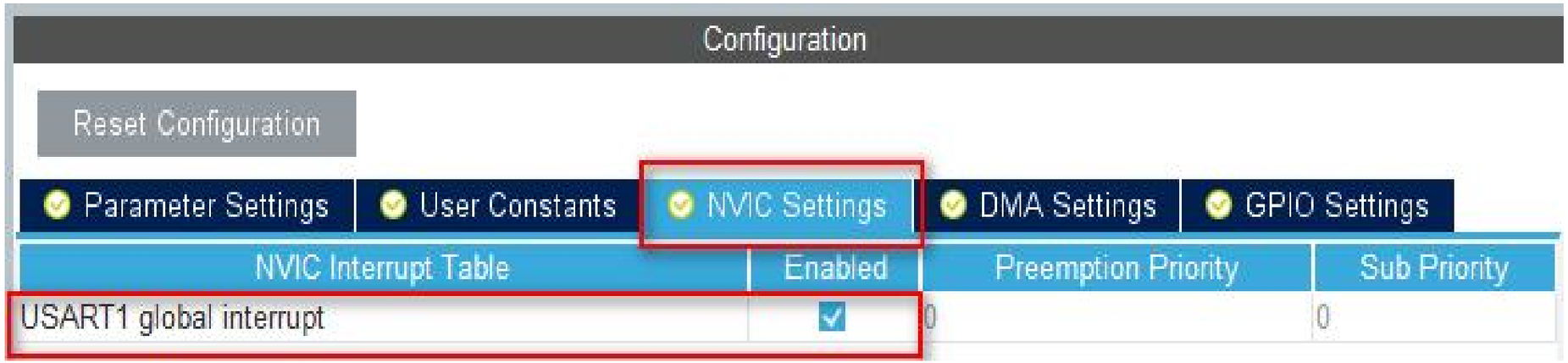


- USART1的发送引脚USART1_Tx默认连接在PA9；
- 接收引脚USART1_Rx默认连接在PA10；
- 在配置USART1时已经将PA9引脚设置为复用推挽输出，PA10设置为输入模式。

软件设计——STM32CubeMX功能参数配置

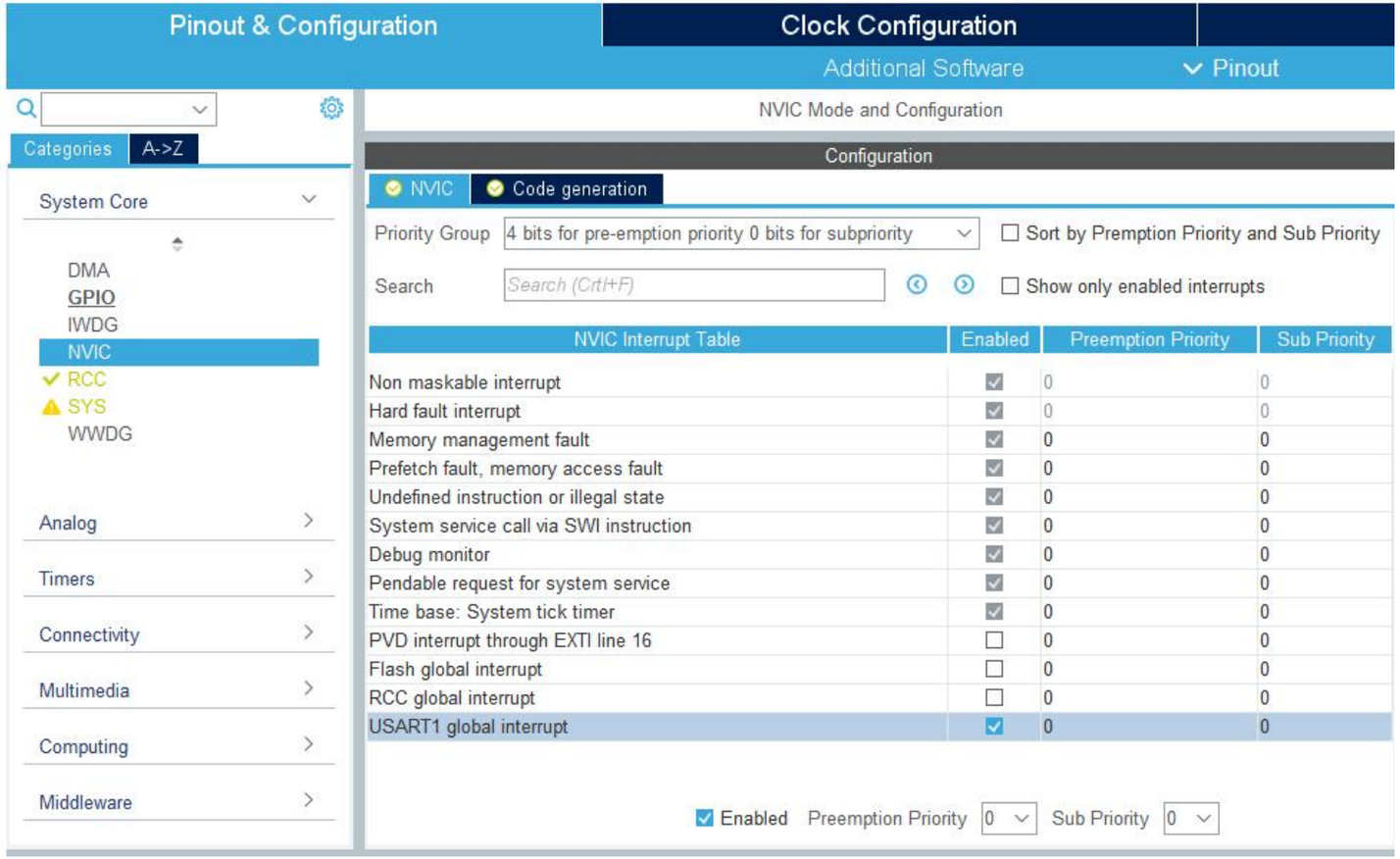
开启串口USART1的中断

- 在“Configeration”配置页面的“NVIC Settings”中勾选“USART1 global interrupt”。



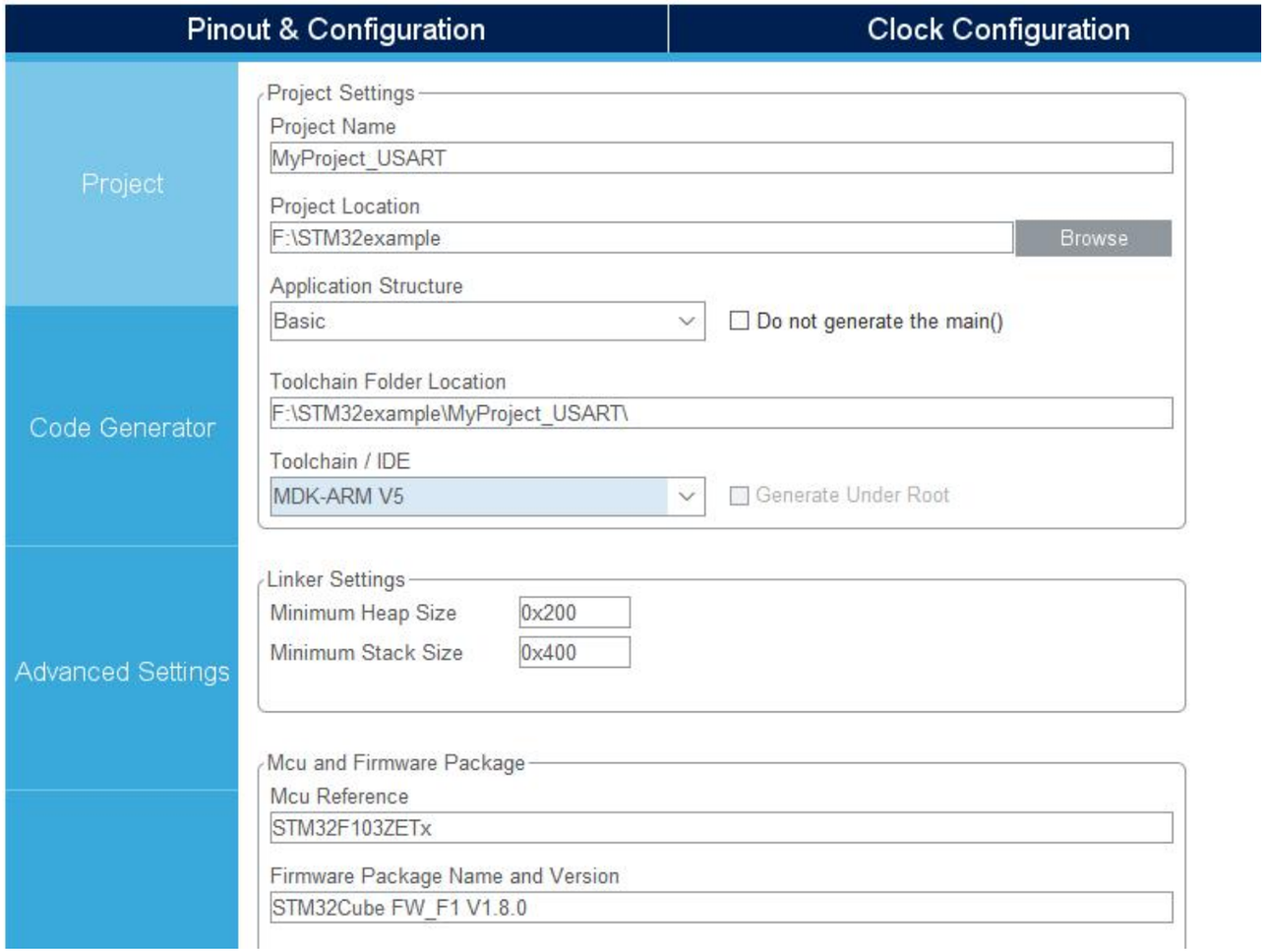
软件设计——STM32CubeMX功能参数配置

选择NVIC中断优先级分组



- 在“System Core”的“NVIC”中可选择NVIC中断优先级分组，默认为“4bits for pre-amption priority”，勾选“USART1 global interrupt”。

软件设计——生成工程代码

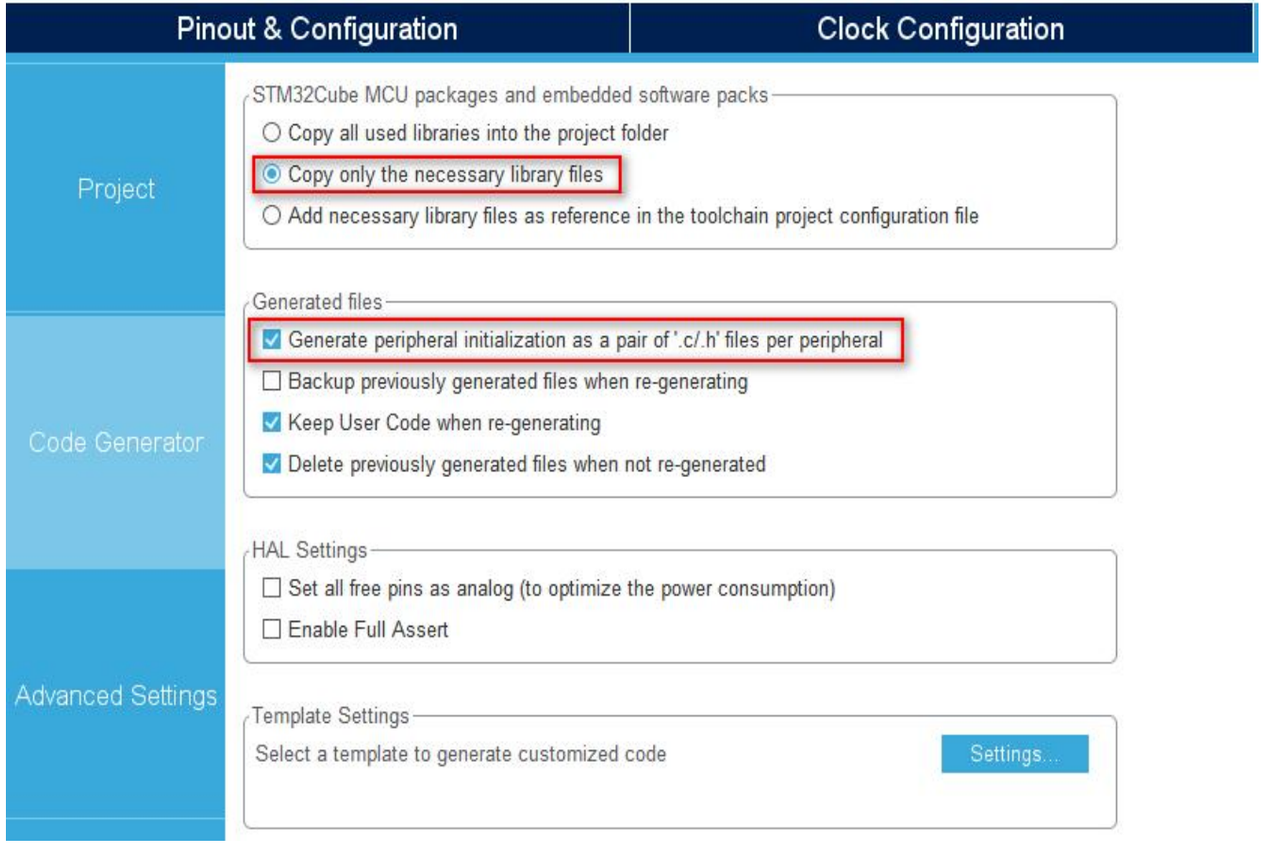


配置keil工程名称和存放位置

配置工程名称、工程保存位置、选择“MDK-ARM V5”编译器等



软件设计——生成工程代码



在“Code Generator”选项栏中找到“Generated files”框，勾选“Generate peripheral initialization as a pair of ’.c/.h’ files per IP”，将外设初始化的代码设置生成为独立的.c源文件和.h头文件。

软件设计——编写应用程序

在main.c文件中添加代码：

- 在main.c文件中添加代码。先定义两个数组RxBuffer和TxBuffer，用于接收数据和发送数据；
- 在main.c文件的/* USER CODE BEGIN PV */和/* USER CODE END PV */之间添加代码

```
/* USER CODE BEGIN PV */
uint8_t RxBuffer[10];
uint8_t TxBuffer[10];
/* USER CODE END PV */
```

软件设计——编写应用程序

在int main(void)函数中的/* USER CODE BEGIN 2 */和/* USER CODE END 2 */之间添加代码：

```
/* USER CODE BEGIN 2 */
/*  使能USART1接收，调用中断处理函数 */
HAL_UART_Receive_IT(&huart1,(uint8_t *)&RxBuffer,1);
/* USER CODE END 2 */
```

中断处理函数进行相应功能程序的处理，在/* USER CODE BEGIN 4 */和/* USER CODE END 4 */之间添加代码：

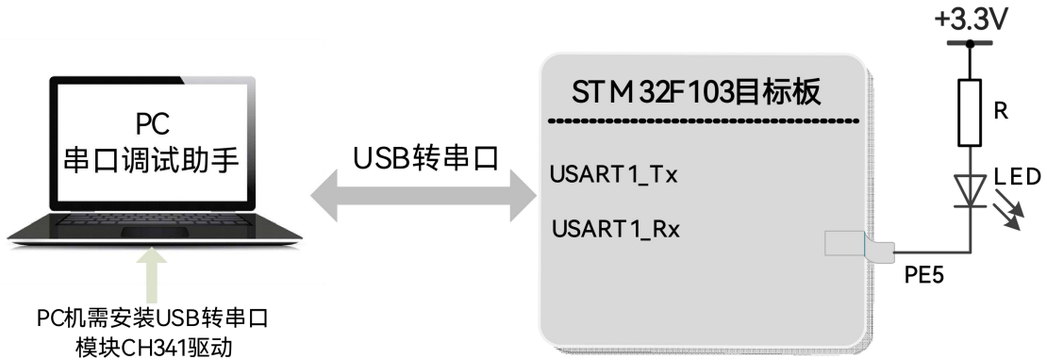
```
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{    HAL_UART_Transmit(&huart1,(uint8_t *)&RxBuffer,1,0);
    HAL_UART_Receive_IT(&huart1,(uint8_t *)&RxBuffer,1);
}
/* USER CODE END 4 */
```

7.5.3 USART应用实例拓展

功能

基于HAL库采用中断控制方式实现上位机（PC）通过串口通信控制STM32目标板上LED灯的亮灭，发送字符命令“Y”点亮LED灯，发送字符命令“N”熄灭LED灯。

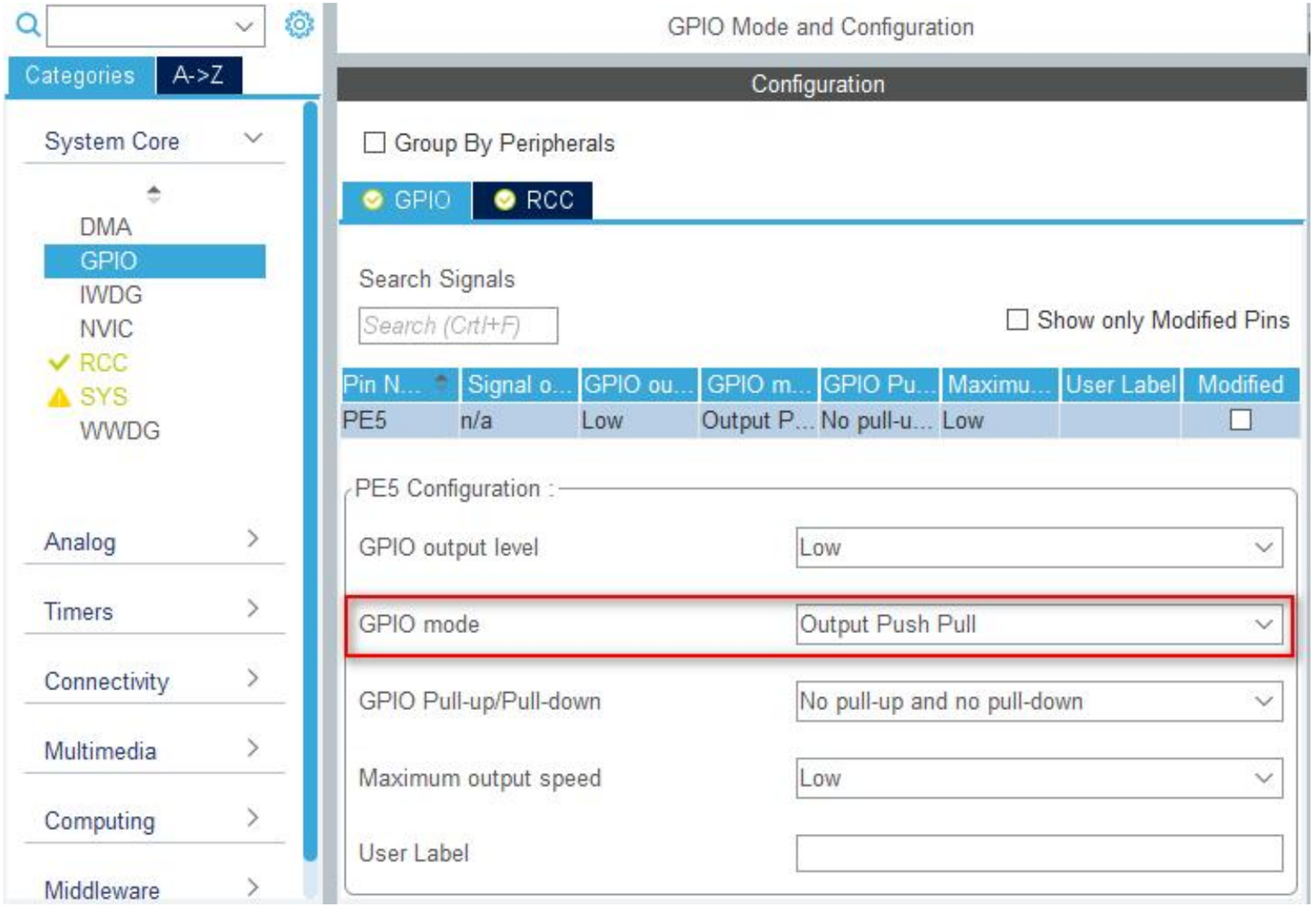
硬件设计



使用STM32F103的串口1（USART1），PE5引脚连接一个LED灯。

7.5.3 USART应用实例拓展

将PE5配置为“Output Push Pull”（推挽输出），初始化电平为低电平



7.5.3 USART应用实例拓展

编写应用程序

首先定义一个字符变量Receive_data用于存放接收的字符，在main.c文件中的while()中添加如下程序代码：

```
while (1)
{
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
uint8_t Receive_data; //定义一个字符变量
Receive_data = getchar(); //获取输入的命令字符
printf("The Receive order is :
%c\n",Receive_data);
if(Receive_data == 'Y')  //发送字符命令Y，点亮LED灯
{
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5,
GPIO_PIN_RESET);
printf("The LED is ON !\r\n");
}
else if(Receive_data == 'N') //发送字符命令N，熄灭LED灯
{
HAL_GPIO_WritePin(GPIOE,GPIO_PIN_5,GPIO_PIN_SET);
printf("The LED is OFF !\r\n");
}
}
/* USER CODE END 3 */
```

7.5.3 USART应用实例拓展

编写应用程序

```
//重定向C库函数scanf()到串口，重定向后可使用scanf()函数
int fgetc(FILE * f)
{
  uint8_t ch = 0;
  HAL_UART_Receive(&huart1,&ch, 1, 0xffff);
  return ch;
}
/* USER CODE END 1 */
```

重新编译工程，无错误无警告后，下载到目标开发板进行验证。

7.6 编程思想之自定义串口通信协议

复杂的通信控制往往是以数据帧为单位进行传输的。帧是数据传输的一种单位，一帧数据由多个字符组合而成，不同字段的字符代表不同的含义，实现不同功能。

自定义串口通信协议就是根据需求定义一个数据帧的格式

发送端

按照规定的数据帧格式发送一帧数据

接收端

根据规定的数据帧格式进行解析，获取正确的数据。

7.6 编程思想之自定义串口通信协议

数据帧格式

起始位	数据1	数据2	校验位	结束位
0xAA	01：设备1 02：设备2	00：功能1 01：功能2	——	0xCC
0xBB	01：设备1 02：设备2	02：功能1 03：功能2	——	0xCC

起始位：也就是帧头，表示一帧数据的开始，必须要有，长度不限，但不建议太长。

数据1：用于表示设备，长度不限，用户根据需要自定义。

数据2：用于功能选择，长度不限，用户根据需要自定义。

校验位：复杂的通信协议中需要用到，一些简单的自定义串口通信协议此项可有可无，长度一般为1～2个字节。

结束位：也即帧尾，表示一帧数据的结束，长度不限。

7.6 编程思想之自定义串口通信协议

应用实例：上位机通过一个由4个字节组成的数据帧控制指示灯LED的亮灭

帧头	数据1	数据2	帧尾
0xAA	0xa1指示灯	0x01指示灯亮	0x55
		0x02指示灯灭	

在主程序中定义一个数组变量，用于接收数据

```
/* USER CODE BEGIN PV */
  uint8_t Rx_Buff[16];
      // 定义一个数组，用于数据接收
/* USER CODE END PV */
```

开启串口接收中断

```
  /* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart1,
Rx_Buff,4);
  /* USER CODE END 2 */
```

7.6 编程思想之自定义串口通信协议

在串口中断回调函数中，根据接收的数据内容进行相应的中断操作

```
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if(huart->Instance == USART1)  //判断是哪个串口触发的接收中断
  {  if(Rx_Buff[0] == 0xAA && Rx_Buff[3] == 0x55) //判断帧头和帧尾
     {    switch(Rx_Buff[1])
           {
               case 0x01:  switch(Rx_Buff[2])
                  {     case 0x01:HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin,
GPIO_PIN_RESET);
                        printf("LED2 is ON! \r\n");
                                    break;
                         case 0x02: HAL_GPIO_WritePin(LED2_GPIO_Port,LED2_Pin,
GPIO_PIN_SET);
                        printf("LED2 is OFF !\r\n");
                        break;
                        default:   printf("指令错误!\r\n");
                    break;
                }
```

7.6 编程思想之自定义串口通信协议

在串口中断回调函数中，根据接收的数据内容进行相应的中断操作

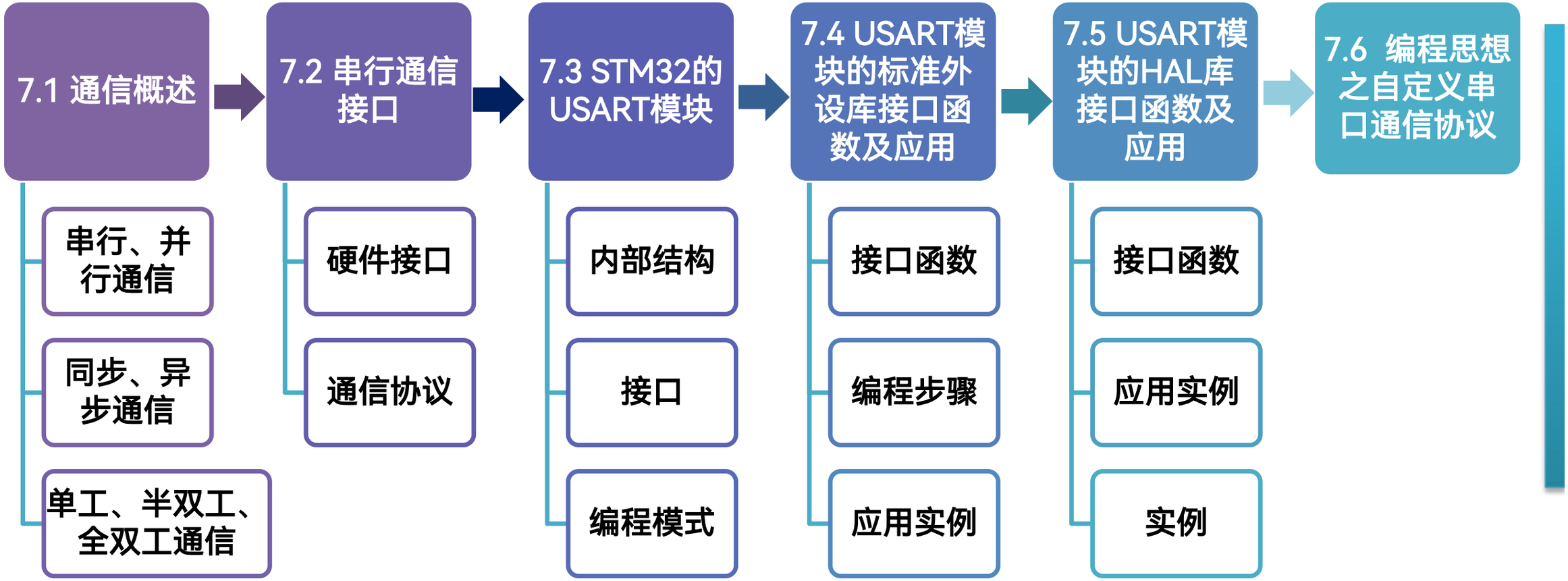
```
                    break;
                default:
                    printf("The command is wrong !\r\n");
                    break;
            }
        HAL_UART_Receive_IT(&huart1,Rx_Buff,4);
    }
    else
    {
        printf("命令错误!\r\n");
    }
    HAL_UART_Receive_IT(&huart1,Rx_Buff,4);
  }
}
/* USER CODE END 4 */
```

7.6 编程思想之自定义串口通信协议

程序对数据帧的处理放在中断回调函数中。中断回调函数：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef
*huart)
{
    if(huart->Instance==USART1)
    {
        Rx_Flag=1;
         HAL_UART_Receive_IT(&huart1,(uint8_t
*)Rx_Buff,4);
    }
}
```

本章小结


7.1 通信概述
串行、并行通信
同步、异步通信
单工、半双工、全双工通信
7.2 串行通信接口
硬件接口
通信协议
7.3 STM32的USART模块
内部结构
接口
编程模式
7.4 USART模块的标准外设库接口函数及应用
接口函数
编程步骤
应用实例
7.5 USART模块的HAL库接口函数及应用
接口函数
应用实例
实例
7.6 编程思想之自定义串口通信协议



The End!

